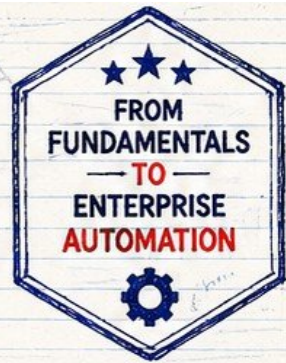




VERIQTA

PRODUCTION-READY • AUTOMATE EVERYTHING • SCALE ANYWHERE



ANSIBLE HANDBOOK

FROM FUNDAMENTALS TO ENTERPRISE AUTOMATION



AGENTLESS AUTOMATION

No agents.
No complexity.



INFRASTRUCTURE AS CODE

Declarative.
Versioned. Repeatable.



CONFIGURATION MANAGEMENT

Consistent systems.
Every time.



MULTI-CLOUD AUTOMATION

AWS, Azure, GCP
and beyond.



SECURE & COMPLIANT

Built-in security.
Enterprise ready.



EASY TO LEARN POWERFUL

Simple YAML.
Infinite possibilities.



ROLES & COLLECTIONS

Reusable.
Shareable.



INTEGRATE EVERYWHERE

CI/CD, ITSM,
Monitoring & More.



PRODUCTION GRADE

Built for scale.
Built for reliability.



SCALE AUTOMATION

From servers to
entire enterprises.



AUTOMATION IS NOT A LUXURY.
IT'S A SUPERPOWER.



AUTOMATE ANYTHING

From simple tasks
to complex workflows.



REDUCE RISK AND ERROR

Consistency and compliance
at enterprise scale.



SAVE TIME DELIVER FASTER

Focus on innovation,
not repetition.



BUILT FOR ENGINEERS

By engineers.
For engineers.





INTRODUCTION TO ANSIBLE



ANSIBLE

“ Ansible is simple IT automation.
Powerful. Agentless. Human readable.

VERIQTA ”

1 WHAT ANSIBLE IS



Ansible is an open source automation engine that helps you configure systems, deploy applications, and orchestrate infrastructure – all in a simple, agentless way.

2 WHY ENTERPRISES USE ANSIBLE



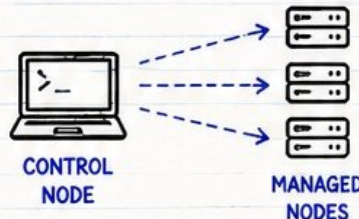
- ✓ Reduces manual work and errors
- ✓ Consistent and repeatable automation
- ✓ Speeds up deployments
- ✓ Improves compliance and security
- ✓ Easy to learn and maintain
- ✓ Scales across environments

3 AGENTLESS AUTOMATION



Ansible does not require any software or agent to be installed on managed nodes. It uses SSH (or WinRM for Windows) to connect and automate.

4 PUSH ARCHITECTURE



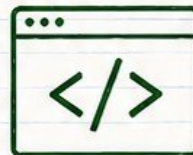
Ansible uses a push model. The control node connects to managed nodes and pushes tasks to execute. No inbound connections are required.

5 COMMON USE CASES

- Configuration Management
- Application Deployment
- Provisioning Infrastructure
- Cloud Management
- Continuous Delivery
- Security & Compliance
- Network Automation



6 INFRASTRUCTURE AS CODE



Ansible uses YAML playbooks to define infrastructure, applications, and configurations as code. Version it. Review it. Reuse it. Scale it.

7 AUTOMATION MINDSET



- ✓ Automate everything that can be automated.
- ✓ Make processes repeatable and reliable.
- ✓ Build for scale, consistency, and security.
- ✓ Continuously improve and eliminate manual work.

THINK AUTOMATION FIRST

Every task you automate today saves time, reduces risk, and builds a stronger future.

VERIQTA

★ **PRO TIP:** Start small. Automate one task. Prove value. Then automate everything.

VERIQTA



YouTube
VERIQTA



GitHub
veriqta



LinkedIn
VERIQTA



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



ANSIBLE

ANSIBLE ARCHITECTURE

“ Ansible is agentless. It connects over SSH and pushes automation from the control node to managed nodes.

— VERIQTA ”

1 CONTROL NODE



- ✓ The machine where Ansible is installed.
- ✓ Orchestrates and pushes tasks to managed nodes.
- ✓ Runs playbooks and modules.
- ✓ Stores inventory, roles, and configurations.

2 MANAGED NODES



- ✓ Target machines (servers, devices, cloud instances, containers, etc.).
- ✓ No agents or software required.
- ✓ Receive tasks from the control node and return results.

3 INVENTORY



- ✓ Defines the managed hosts and groups.
- ✓ Static (INI/YAML) or Dynamic (from cloud, CMDB, scripts).
- ✓ Can include host variables and group variables.

4 SSH COMMUNICATION



- ✓ Ansible connects to managed nodes using SSH (port 22).
- ✓ Secure, encrypted, and reliable.
- ✓ Uses key-based authentication by default.

5 MODULES



- ✓ Reusable units of code that perform specific tasks.
- ✓ Built-in modules (File, Copy, Service, User, Package, etc.).
- ✓ Can be used in ad-hoc commands or playbooks.

6 PLUGINS



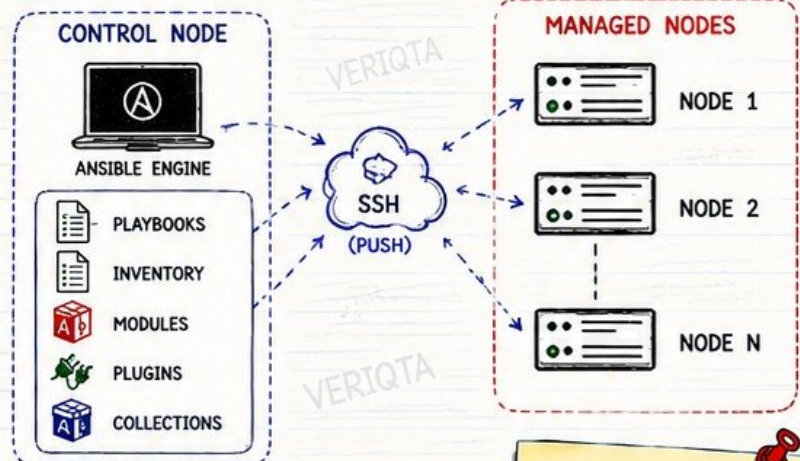
- ✓ Extend Ansible's core capabilities.
- ✓ Types: Connection, Inventory, Lookup, Callback, Action, Cache, etc.
- ✓ Enable integrations and customization.

7 COLLECTIONS



- ✓ Distributable units of Ansible content.
- ✓ Include modules, plugins, roles, playbooks, and docs.
- ✓ Installed from Galaxy or private repositories.

8 EXECUTION WORKFLOW



WORKFLOW STEPS

1. Ansible reads inventory and playbook.
2. Connects to managed nodes over SSH.
3. Sends modules (tasks) for execution.
4. Modules execute on nodes.
5. Results are returned to the control node.
6. Ansible reports status and changes.

KEY POINT

Ansible does not pull. It pushes automation from the control node to the managed nodes in a secure and agentless way.

VERIQTA

★ PRO TIP: Keep your control node secure, your inventory accurate, and your playbooks idempotent.

VERIQTA

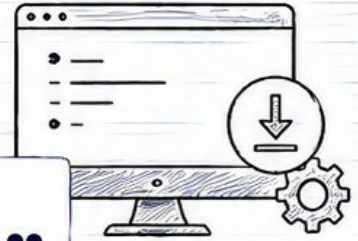


INSTALLING ANSIBLE



“ A solid installation is the foundation of reliable automation.
Install it right. Automate everything.

— VERIQTA ”

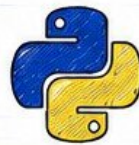


1 INSTALLATION METHODS



- OS package managers (apt, yum, dnf, zypper)
- Python package manager (pip)
- Source installation
- Pre-built execution environments
- Ansible Automation Platform (Enterprise)

2 PYTHON REQUIREMENTS



- ✓ Python 3.8 or higher (recommended)
- ✓ Python 3.10+ for best compatibility
- ✓ pip 20.3+
- ✓ setuptools
- ✓ Jinja2
- ✓ PyYAML
- ✓ cryptography

```
-- CHECK PYTHON
$ python3 --version
Python 3.10.12
$ pip3 --version
pip 23.2.1
```

3 PACKAGE MANAGERS



Ubuntu /
Debian (apt) →

```
$ sudo apt update
$ sudo apt install ansible
```



RHEL / CentOS /
Rocky (yum/dnf) →

```
$ sudo dnf install ansible
# or on older systems
$ sudo yum install ansible
```



SUSE (zypper) →

```
$ sudo zypper install ansible
```

4 VIRTUAL ENVIRONMENTS (RECOMMENDED)



```
$ python3 -m venv ansible-venv
$ source ansible-venv/bin/activate
(ansible-venv) $ pip install --upgrade pip
(ansible-venv) $ pip install ansible
(ansible-venv) $ deactivate # exit venv
```



WHY USE VIRTUAL ENV?

- Isolate dependencies
- Avoid conflicts
- Easier upgrades and rollbacks
- Cleaner, reproducible environments

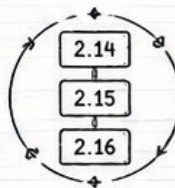
5 VERIFYING INSTALLATION



```
$ ansible --version
ansible [core 2.16.6]
config file = /etc/ansible/ansible.cfg
configured module search path = [...]
python python module location = [...]
python version = 3.10.12 (default, ...)
```

✓ Also test connectivity:
\$ ansible all -m ping

6 VERSION MANAGEMENT



- ✓ Use consistent Ansible versions across teams.
- ✓ Track versions in requirements.txt or via automation tools.
- ✓ Test playbooks before upgrading.
- ✓ Read CHANGELOG before major upgrades.

```
# requirements.txt
ansible=2.16.6
```

7 ENTERPRISE RECOMMENDATIONS



- ✓ Use virtual environments or execution environments.
- ✓ Pin versions for stability in production.
- ✓ Store requirements.txt in source control.
- ✓ Use Ansible Automation Platform for enterprise needs.
- ✓ Automate installation via CI/CD or configuration management.
- ✓ Apply security updates regularly.
- ✓ Document and standardize your installation process.

PRO TIP

A predictable installation
leads to predictable
automation. Standardize it.
Version it. Automate it.

— VERIQTA

QUICK REFERENCE

OS Packages → Easy, system-wide
pip → Flexible, user-space
venv → Isolated, recommended
AAP → Enterprise ready

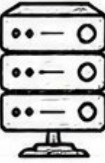
★ PRO TIP: Keep Ansible updated, but never upgrade without testing in a staging environment.

VERIQTA



VERIQTA

INVENTORY MANAGEMENT



“ A well-structured inventory is the foundation of reliable and scalable automation. ”

— VERIQTA

1 STATIC INVENTORY



- ✓ Traditional INI/YAML files.
- ✓ Simple and easy to understand.
- ✓ Stored on the control node.
- ✓ Best for small to medium environments.

```
[webservers]
web1 ansible_host=10.0.1.10
web2 ansible_host=10.0.1.11

[dbservers]
db1 ansible_host=10.0.2.10
```

2 DYNAMIC INVENTORY



- ✓ Generated in real time from external sources.
- ✓ Supports cloud, CMDB, virtualization, etc.
- ✓ Always reflects the current environment.

Examples:

- AWS EC2, Azure, GCP
- VMware vCenter
- NetBox, OpenStack
- Scripted inventory

3 INVENTORY GROUPS

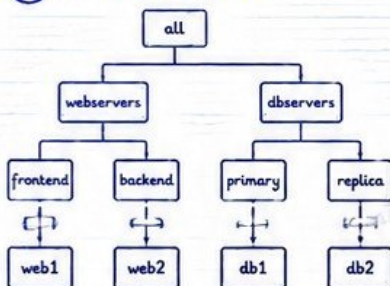


- ✓ Organize hosts into logical groups.
- ✓ Use groups for targeting and variables.
- ✓ Hosts can belong to multiple groups.

```
[webservers]
web1
web2
[dbservers]
db1
[all:children]
webservers
dbservers
```

Use groups to run plays against multiple hosts.

4 NESTED GROUPS



- ✓ Groups can contain sub-groups.
- ✓ Use :children to define relationships.

```
[frontend]
web1
[backend]
web2
[webservers:children]
frontend
backend
```

5 HOST VARIABLES



- ✓ Variables specific to a single host.
- ✓ Defined in inventory or separate files.
- ✓ Override group variables.

```
web1 ansible_host=10.0.1.10
ansible_user=ubuntu
ansible_port=22
app_env=production
app_version=1.2.0
```

EXAMPLE (host_vars/web1.yml)

```
ansible_user: ubuntu
ansible_port: 22
app_env: production
app_version: 1.2.0
```

6 GROUP VARIABLES



- ✓ Variables for all hosts in a group.
- ✓ Defined in inventory or group_vars/ directory.
- ✓ Lower precedence than host vars.

EXAMPLE (group_vars/webservers.yml)

```
ansible_user: ubuntu
app_env: staging
http_port: 80
log_level: info
```

7 INVENTORY BEST PRACTICES



- ✓ Keep inventory organized and consistent.
- ✓ Use meaningful group and host names.
- ✓ Use variables to avoid repetition.
- ✓ Prefer dynamic inventory for cloud environments.
- ✓ Store variables in group_vars/ and host_vars/.
- ✓ Use nested groups to reflect real-world structure.
- ✓ Document your inventory structure.
- ✓ Test inventory changes before production.

PRO TIP



A clean inventory makes automation scalable, maintainable, and predictable.

VERIQTA

INVENTORY FILE LOCATIONS



```
/etc/ansible/hosts
./inventory/hosts.ini
./inventory/hosts.yml
Dynamic source (cloud, CMDB, scripts)
```

INVENTORY COMMANDS



```
ansible-inventory --list
ansible-inventory --graph
ansible-inventory --host <hostname>
ansible all -m ping -i inventoryfile
```

★ PRO TIP: Good inventory design today saves hours of troubleshooting tomorrow.

VERIQTA



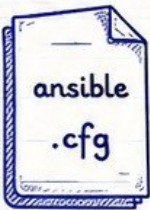
ANSIBLE CONFIGURATION



“ A good configuration makes automation faster, safer, and more reliable.

— VERIQTA ”

1 ANSIBLE.CFG

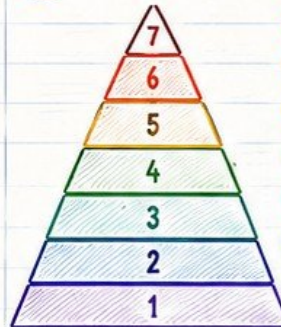


- ✓ Main configuration file for Ansible.
- ✓ Controls defaults for behavior, modules, inventory, SSH, performance, and more.
- ✓ Located in current directory, home (~/.ansible.cfg), or /etc/ansible/ansible.cfg

Example ansible.cfg

```
[defaults]
inventory = ./inventory/hosts.ini
remote_user = ansible
host_key_checking = False
retry_files_enabled = False
stdout_callback = yaml
forks = 20
gathering = smart
fact_caching = jsonfile
fact_caching_timeout = 86400
```

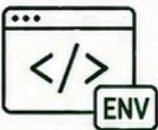
2 CONFIGURATION PRECEDENCE



- 7 Extra vars (-extra-vars, -e)
- 6 Task vars (in playbooks)
- 5 Block vars
- 4 Role vars
- 3 Inventory vars (host/group vars)
- 2 ansible.cfg settings
- 1 Built-in defaults

Higher number = Higher precedence

3 ENVIRONMENT VARIABLES



- ✓ Environment variables can override ansible.cfg settings.
- ✓ Useful for temporary changes or CI/CD.
- ✓ Common Ansible environment variables:

```
ANSIBLE_CONFIG          # Custom config path
ANSIBLE_INVENTORY       # Inventory file/path
ANSIBLE_HOST_KEY_CHECKING # True/False
ANSIBLE_STDOUT_CALLBACK # Output format
ANSIBLE_FORKS           # Parallel processes
ANSIBLE_REMOTE_USER     # SSH user
ANSIBLE_TIMEOUT         # SSH timeout
```

4 DEFAULT SETTINGS (EXAMPLES)



Key default settings in [defaults]

```
inventory = /etc/ansible/hosts
remote_user = (current user)
forks = 5
timeout = 10
host_key_checking = True
gathering = implicit
fact_caching = memory
retry_files_enabled = True
stdout_callback = default
```

TIP

Always customize these defaults to suit your environment for security and performance.

— VERIQTA

5 SSH CONFIGURATION



- ✓ Ansible uses SSH to connect to managed nodes.
- ✓ Control SSH behavior via ansible.cfg or ~/.ssh/config.
- ✓ Best practices:
 - ✓ Use key-based authentication
 - ✓ Disable password authentication
 - ✓ Use dedicated automation users
 - ✓ Enforce strict host key checking in production

Example ~/.ssh/config

```
Host *
  ServerAliveInterval 30
  ServerAliveCountMax 3
  StrictHostKeyChecking yes
  UserKnownHostsFile ~/.ssh/known_hosts
```

6 PERFORMANCE TUNING



- ✓ Increase forks for parallelism.
- ✓ Use fact caching to reduce load.
- ✓ Limit gathering to needed facts.
- ✓ Disable unnecessary callbacks.
- ✓ Use persistent SSH connections.
- ✓ Optimize modules and playbooks.

Example tuning in ansible.cfg

```
[defaults]
forks = 50
gathering = smart
fact_caching = jsonfile
fact_caching_timeout = 86400
ssh_args = -o ControlMaster=auto -o ControlPersist=60s
```

7 ENTERPRISE STANDARDS



- ✓ Maintain version-controlled ansible.cfg.
- ✓ Use consistent inventory structure.
- ✓ Enforce security: SSH keys, no passwords.
- ✓ Standardize variable management.
- ✓ Use collections and FQCNs.
- ✓ Document and audit configuration changes.

Recommended Enterprise ansible.cfg Structure

```
/etc/ansible/ansible.cfg # Global defaults
/etc/ansible/hosts.ini   # Main inventory
/etc/ansible/group_vars/ # Group variables
/etc/ansible/host_vars/  # Host variables
/etc/ansible/roles/      # Roles directory
/etc/ansible/collections/ # Collections
```

BEST PRACTICE

Keep configuration consistent, documented, secure, and optimized for scale. Automation at scale depends on standards.

— VERIQTA

★ PRO TIP: Good configuration today prevents outages tomorrow.

VERIQTA



ANSIBLE



Quick.
Powerful.
Effective.

VERIQTA AD HOC COMMANDS



Ad hoc commands are one-liners used to run modules without writing a playbook.

- VERIQTA

“Ad hoc commands are perfect for quick tasks, troubleshooting, and day-to-day operations.”

- VERIQTA

1 PING MODULE



- ✓ Check connectivity to managed nodes.
- ✓ Uses ICMP echo (like ping).

```
# Ping all hosts
ansible all -m ping
```

```
# Ping a specific group
ansible webservers -m ping
```

Sample Output

```
web1 | SUCCESS => {
  "ping": "pong"
}
web2 | SUCCESS => {
  "ping": "pong"
}
```

2 SHELL MODULE



- ✓ Runs commands through the system shell.
- ✓ Use when shell features are needed (pipes, redirects, env vars).
- ✓ Not idempotent.

```
# List disk usage
ansible all -m shell -a "df -h"
```

```
# Check running processes
ansible all -m shell -a "ps aux | grep nginx"
```

3 COMMAND MODULE



- ✓ Executes commands without using the shell.
- ✓ Faster and safer than shell.
- ✓ Use for simple commands.

```
# Check uptime
ansible all -m command -a "uptime"
```

```
# Get OS release info
ansible all -m command -a "cat /etc/os-release"
```

4 FILE OPERATIONS



- ✓ Manage files and directories.
- ✓ Common modules: file, copy, fetch, move, delete.
- ✓ Idempotent and safe.

```
# Create directory
ansible all -m file -a "path=/opt/data state=directory mode=0755"
```

```
# Copy file
ansible all -m copy -a "src=app.conf dest=/etc/app.conf"
```

```
# Remove file
ansible all -m file -a "path=/tmp/old.log state=absent"
```

5 USER MANAGEMENT



- ✓ Manage system users and groups.
- ✓ Common modules: user, group.
- ✓ Idempotent by default.

```
# Create user
ansible all -m user -a "name=devops shell=/bin/bash state=present"
```

```
# Add user to sudo group
ansible all -m user -a "name=devops groups=sudo append=yes"
```

6 PACKAGE INSTALLATION



- ✓ Install, update or remove packages.
- ✓ Module: apt (Debian/Ubuntu), yum/dnf (RHEL/CentOS), package (generic).

```
# Install nginx (Debian/Ubuntu)
ansible all -m apt -a "name=nginx state=present update_cache=yes"
```

```
# Install httpd (RHEL/CentOS)
ansible all -m yum -a "name=httpd state=present"
```

7 PRODUCTION EXAMPLES



Check server status
(fast reachability test)

```
→ ansible all -m ping
```



Check disk space on all servers

```
→ ansible all -m shell -a "df -h"
```



Show memory usage

```
→ ansible all -m command -a "free -m"
```



Create a directory on web servers

```
→ ansible webservers -m file -a "path=/var/www/html state=directory mode=0755"
```



Create deploy user

```
→ ansible all -m user -a "name=deploy shell=/bin/bash state=present"
```



Install git package

```
→ ansible all -m package -a "name=git state=present"
```



Clean temporary files

```
→ ansible all -m file -a "path=/tmp/* state=absent"
```

BEST PRACTICES

- ✓ Test ad hoc commands before automation.
- ✓ Use `--limit` for targeted execution.
- ✓ Prefer command module over shell when possible.
- ✓ Keep commands idempotent.
- ✓ Use `--become` for privileged tasks.
- ✓ Log and document all operations.



PRO TIP: Ad hoc commands are powerful. Use them wisely and document important one-liners for your team. - VERIQTA



ANSIBLE

Modules are the building blocks of Ansible.

UNDERSTANDING MODULES

“Modules are the heart of Ansible. They perform the work on managed nodes in an idempotent and reliable way.”

- VERIQTA ”

WHAT IS A MODULE?

A module is a reusable unit of code that Ansible uses to communicate with managed nodes and perform tasks.

1 BUILT-IN MODULES



- ✓ Ansible comes with 1000+ built-in modules.
- ✓ No installation required.
- ✓ Covers system, services, files, packages, users, networking, cloud and more.
- ✓ Always check docs before using any module.

Example: Using the ping module

```
ansible all -m ping
```

Output

```
web1 | SUCCESS => {"ping": "pong"}
db1  | SUCCESS => {"ping": "pong"}
```

2 IDEMPOTENCY



- ✓ Modules are idempotent by design.
- ✓ Running a task multiple times produces the same result.
- ✓ Changes are made only when required.
- ✓ Essential for reliable automation.

Example: Create a file (run multiple times)

- name: Create a file

file:

path: /opt/app/data.txt
state: touch

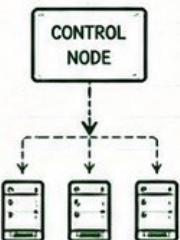
Run 1

Run 2

changed: true

changed: false

3 MODULE EXECUTION



- ✓ Control node sends module to managed node.
- ✓ Module is executed on the remote node.
- ✓ Result is returned to control node.
- ✓ No agents installed on managed nodes.

Execution Flow

1. Ansible connects via SSH
2. Module code is transferred
3. Module is executed
4. Result is returned
5. Ansible processes the result

4 LINUX MODULES (Examples)



MODULE	PURPOSE
file	Manage files and directories
copy	Copy files to managed nodes
template	Deploy templates using Jinja2
user	Manage users and groups
service	Manage system services
package	Install or remove packages
command	Run commands (no shell features)
shell	Run shell commands

5 CLOUD MODULES (Examples)



MODULE	PURPOSE
ec2	Manage AWS EC2 instances
s3	Manage AWS S3 buckets
iam	Manage AWS IAM resources
rds	Manage AWS RDS databases
azure_rm_resourcegroup	Manage Azure Resource Groups
gcp_compute_instance	Manage GCP Compute Instances

6 NETWORK MODULES (Examples)



MODULE	PURPOSE
ios_config	Manage Cisco IOS config
ios_command	Run Cisco IOS commands
nxos_config	Manage Cisco NX-OS config
junos_config	Manage Juniper Junos config
eos_config	Manage Arista EOS config
net_get	Gather facts from network devices

7 MODULE DOCUMENTATION



- ✓ Always read module documentation.
- ✓ Understand options, defaults, and examples.
- ✓ Documentation is available locally and online.
- ✓ Use docs - it saves time and prevents errors.

WHERE TO FIND DOCS

1. `ansible-doc -l` # List all modules
2. `ansible-doc <module_name>` # View module docs
3. `docs.ansible.com` # Official documentation
4. Galaxy documentation # Role and collection docs

Example

```
$ ansible-doc copy
$ ansible-doc -l | grep ec2
$ ansible-doc service | less
```



PRO TIP: Use the right module for the job. It makes playbooks cleaner, faster, and more reliable.

VERIQTA



ANSIBLE

YAML is the language that makes Ansible playbooks powerful, readable and scalable.

YAML FOR ANSIBLE

“Great playbooks start with clean, correct and well-structured YAML.”

— VERIQTA ”

YAML PRINCIPLES

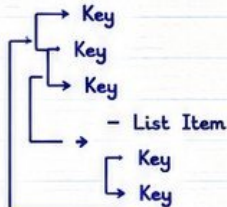
- ✓ Human readable
- ✓ Indentation sensitive
- ✓ Consistent structure
- ✓ Predictable behavior
- ✓ Perfect for automation

1 YAML SYNTAX

- ✓ Indentation (spaces, not tabs) defines structure.
- ✓ Use colon (:) to separate key and value.
- ✓ Indent with 2 spaces (recommended).
- ✓ Sensitive to whitespace.

EXAMPLE

```
name: Install Nginx
hosts: webservers
become: yes
tasks:
  - name: Install nginx package
    apt:
      name: nginx
      state: present
```



2 LISTS

- ✓ Lists start with a dash (-).
- ✓ Items are aligned under the list key.
- ✓ Used for tasks, servers, packages, etc.

EXAMPLE: LIST OF PACKAGES

```
packages:
  - nginx
  - git
  - curl
  - htop
```

List Structure



3 DICTIONARIES

- ✓ Key: value pairs.
- ✓ Used for variables, module arguments, etc.
- ✓ Keys must be unique.

EXAMPLE: DICTIONARY

```
user:
  name: devops
  shell: /bin/bash
  state: present
  groups: sudo
```

Dictionary Structure

```
key: value
key: value
key: value
key: value
```

4 VARIABLES

- ✓ Variables start with a name and value.
- ✓ Use anywhere in playbooks with {{ variable }}.
- ✓ Supports strings, numbers, lists, dictionaries.

EXAMPLE: VARIABLES

```
vars:
  app_name: nginx
  app_port: 80
  app_version: "1.25.2"
  servers:
    - web1
    - web2
```

Use in Playbook

```
name: Deploy {{ app_name }}
service:
  name: "{{ app_name }}"
  state: started
  port: "{{ app_port }}"
```

5 MULTI-LINE STRINGS

- ✓ Use | for literal block (preserves new lines).
- ✓ Use > for folded block (converts new lines to spaces).
- ✓ Great for scripts, configs, certificates, etc.

LITERAL (|)

```
script: |
  echo "Hello"
  echo "World"
  echo "Done"
```

New lines are kept

FOLDED (>)

```
message: >
  This is
  a multi-line
  message.
```

New lines become spaces

6 ANCHORS & ALIASES

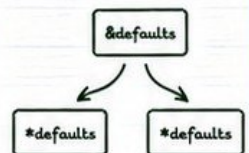
- ✓ Use & to create an anchor.
- ✓ Use * to reuse the anchor.
- ✓ Helps avoid repetition and keep YAML DRY.

EXAMPLE: ANCHORS

```
defaults: &defaults
file1:
  <<: *defaults
  path: /opt/app/file1
owner: root
group: root
```

EXAMPLE: USE ALIAS

```
file1:
  <<: *defaults
  path: /opt/app/file1
file2:
  <<: *defaults
  path: /opt/app/file2
```



7 COMMON MISTAKES

- ✗ Indentation Errors
Wrong indent breaks YAML.

```
tasks:
  - name: bad
    debug:
```

✗

- ✗ Tabs Instead of Spaces
YAML requires spaces only.

```
tasks:
  - name: bad
```

✗

- ✗ Missing Colon
Key value separation requires colon.

```
name "Install Nginx"
```

✗

- ✗ Wrong List Format
Missing dash (-) creates errors.

```
packages:
  nginx
  git
```

✗

- ✗ Quotes in Wrong Place
Unnecessary quotes can cause issues.

```
port: "80"
```

✗

- ✗ Incorrect Anchors
Aliases must match existing anchors.

```
<<: *defaults
```

✗

★ PRO TIP: Write clean YAML. Your future self and your team will thank you. Small mistakes can cause big failures. — VERIQTA



ANSIBLE

PLAYBOOKS

“ Playbooks are where you define what happens, on which systems, and in what order.

- VERIQTA ”

WHAT IS A PLAYBOOK?

A playbook is a YAML file that defines a series of plays and tasks to automate configuration and operations.

1 PLAY STRUCTURE

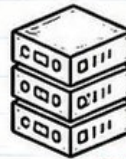


PLAYBOOK COMPONENTS

- ✓ Plays
- ✓ Hosts
- ✓ Tasks
- ✓ Variables
- ✓ Handlers

```
- name: Install and configure Nginx
  hosts: webservers
  become: yes
  vars:
    app_port: 80
  tasks:
    - name: Install Nginx
      apt:
        name: nginx
        state: present
    - name: Start Nginx
      service:
        name: nginx
        state: started
  handlers:
    - name: Restart Nginx
      service:
        name: nginx
        state: restarted
```

2 HOSTS



- ✓ Define target systems
- ✓ Use inventory groups or individual hosts
- ✓ Hosts can be grouped by environment, role, location, etc.
- ✓ Supports patterns and wildcards

EXAMPLES

```
hosts: webservers
hosts: dbservers
hosts: all
hosts: linux:!dbservers
hosts: "tag_Environment_Production"
```

3 TASKS



- ✓ Tasks are the unit of work in a play.
- ✓ Each task calls a module.
- ✓ Tasks run in order from top to bottom.
- ✓ Fail on error unless ignore_errors is set.

EXAMPLE TASK

```
- name: Ensure Nginx is installed
  apt:
    name: nginx
    state: present
```

4 VARIABLES



- ✓ Store values used in playbooks.
- ✓ Can be defined at play, group, host or extra-vars level.
- ✓ Improve reusability and flexibility.

EXAMPLE

```
vars:
  app_name: myapp
  app_port: 80
```

5 HANDLERS



- ✓ Handlers run only when notified.
- ✓ Used for actions like restarting services.
- ✓ Defined at play level under handlers:.
- ✓ Run after all tasks in the play complete.

EXAMPLE HANDLER

```
handlers:
  - name: Restart Nginx
    service:
      name: nginx
      state: restarted
```

6 EXECUTION ORDER



- ✓ Ansible reads the playbook top to bottom.
- ✓ Gathers facts about hosts (unless skipped).
- ✓ Executes tasks in the order defined.
- ✓ Notifies handlers when needed.
- ✓ Runs handlers once at the end of the play.
- ✓ Moves to the next play.

KEY POINT

Handlers run only once, even if notified multiple times.



7 BEST PRACTICES



- ✓ Keep plays small and focused.
- ✓ Use meaningful names for plays and tasks.
- ✓ Use variables and inventories effectively.
- ✓ Use tags to control task execution.
- ✓ Use handlers for service restarts.
- ✓ Test with --check before running.
- ✓ Keep sensitive data in vaults.

SAMPLE PLAY HEADER

```
- name: Configure Web Servers
  hosts: webservers
  become: yes
  vars_files:
    - vars/common.yml
  tags:
    - web
    - setup
```

8 PRODUCTION TIPS



- ✓ Use version control for playbooks.
- ✓ Use roles for modularity and reuse.
- ✓ Use CI/CD to run playbooks automatically.
- ✓ Use logging and monitoring.
- ✓ Use --check and --diff for safe changes.
- ✓ Document playbooks clearly.



PRO TIP: Great playbooks are simple, readable, and repeatable.

- VERIQTA



ANSIBLE

VERIQTA



VARIABLES

POWERFUL. DYNAMIC. ESSENTIAL.

“ Variables make your playbooks flexible, reusable, and environment-aware. ”

- VERIQTA

WHAT ARE VARIABLES?

Variables store data that you can reuse across playbooks, hosts, groups, and tasks to make your automation dynamic and flexible.

1 VARIABLES



- ✓ Variables store values used in playbooks.
- ✓ They can be defined at many levels.
- ✓ Improve reusability and reduce duplication.
- ✓ Support strings, numbers, lists, dictionaries.
- ✓ Referenced using `{{ variable_name }}`.

EXAMPLE

```
vars:
  app_name: nginx
  app_port: 80
  debug_mode: true
  servers:
    - web1
    - web2
```

USAGE

```
→ {{ app_name }}
→ {{ app_port }}
→ {{ servers[0] }}
```

2 VARIABLE PRECEDENCE

HIGHEST



LOWEST

- 1 Extra vars (-e)
- 2 Task vars (include_vars, set_fact)
- 3 Block vars
- 4 Role vars
- 5 Play vars
- 6 Host facts / registered vars
- 7 Host vars (inventory)
- 8 Group vars (inventory)
- 9 Role defaults
- 10 Inventory vars (INI/YAML)
- 11 Ansible configuration defaults

Higher items override lower items.

3 FACTS



- ✓ Facts are variables automatically discovered about managed nodes.
- ✓ Collected using setup module.
- ✓ Stored under `ansible_facts`.
- ✓ Can be used in playbooks like any variable.

EXAMPLE: Accessing Facts

```
{{ ansible_facts['hostname'] }}
{{ ansible_facts['os_family'] }}
{{ ansible_facts['default_ipv4']['address'] }}
```

COMMON FACTS

- hostname
- os_family
- ip_address
- memory_mb
- distribution
- kernel
- and more...

4 REGISTERED VARIABLES



- ✓ Store the result of a task for later use.
- ✓ Access using the variable name you register.
- ✓ Useful for conditions, debug, loops, etc.

```
- name: Check disk space
  command: df -h /
  register: disk_info

- name: Show disk output
  debug:
    var: disk_info.stdout
```

RESULT STRUCTURE

- stdout
- stderr
- rc
- changed
- failed
- and more...

5 EXTRA VARIABLES (-e)



- ✓ Passed at runtime from the command line.
- ✓ Highest precedence.
- ✓ Great for overrides and dynamic input.
- ✓ Can also be passed as JSON or @file.

EXAMPLES

```
ansible-playbook site.yml -e "app_port=8080"
ansible-playbook site.yml -e '{"env":"prod","debug":true}'
ansible-playbook site.yml -e @vars.json
```

6 HOST VARIABLES



- ✓ Variables defined for a specific host.
- ✓ Stored in inventory (INI/YAML) or separate files.
- ✓ Override group vars for that host.

INVENTORY (INI)

```
[webservers]
web1 ansible_host=10.0.0.1 app_port=80
web2 ansible_host=10.0.0.2 app_port=80
```

INVENTORY (YAML)

```
all:
  hosts:
    web1:
      app_port: 80
    web2:
      app_port: 80
```

7 GROUP VARIABLES



- ✓ Variables defined for a group of hosts.
- ✓ Stored in `group_vars/` directory or inventory.
- ✓ Applied to all hosts in that group.

DIRECTORY STRUCTURE

```
group_vars/
├── all.yml
├── webservers.yml
└── dbservers.yml
```

EXAMPLE (group_vars/webservers.yml)

```
app_user: nginx
app_port: 80
app_env: production
```

8 SECURE VARIABLE MANAGEMENT



- ✓ Never store secrets in plain text.
- ✓ Use Ansible Vault to encrypt sensitive data.
- ✓ Store encrypted vars in files or vaults.
- ✓ Use environment variables for external secrets.

USING ANSIBLE VAULT

```
ansible-vault create secrets.yml
ansible-vault edit secrets.yml
ansible-vault view secrets.yml
ansible-vault encrypt_string 'myp@ss'
```

BEST PRACTICES

- Encrypt all secrets
- Use minimal privileges
- Avoid hardcoding
- Rotate credentials
- Audit access

★ PRO TIP: Use variables everywhere. Hardcode never. Reuse always. Secure always.

- VERIQTA



VERIQTA

FACTS AND GATHERING INFORMATION



WHAT ARE FACTS?

Facts are variables about managed nodes that Ansible automatically discovers and makes available.

“ Facts give you the real-time information you need to make smart decisions in your playbooks. ”
- VERIQTA

1 GATHER_FACTS



- ✓ Controls whether Ansible collects facts.
- ✓ Enabled by default (gather_facts: yes).
- ✓ Disable to improve performance.
- ✓ Can be enabled per play or task.

EXAMPLE

```
- name: Disable fact gathering
  hosts: all
  gather_facts: no

- name: Enable fact gathering
  hosts: all
  gather_facts: yes
```

2 SETUP MODULE



- ✓ setup is the module that gathers facts.
- ✓ Automatically runs before tasks (by default).
- ✓ Can be called explicitly to refresh facts.
- ✓ Collects hundreds of variables about the node.

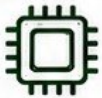
EXAMPLE

```
- name: Gather facts explicitly
  ansible.builtin.setup:
    gather_subset: all
```

COMMON PARAMETERS

- gather_subset: all, min, network, hardware, virtual, oai, !<subset>
- filter: pattern to include/exclude
- fact_path: custom fact path

3 HARDWARE FACTS



- ✓ Information about CPU, memory, disks, BIOS, etc.
- ✓ Useful for sizing, capacity planning, and validation.
- ✓ Available under ansible_facts.hardware.*

EXAMPLES

- ansible_facts.processor_vcpus
- ansible_facts.memtotal_mb
- ansible_facts.devices (disk information)
- ansible_facts.product_name (BIOS/VM info)

4 NETWORK FACTS



- ✓ Information about interfaces, IPs, MACs, gateways.
- ✓ Essential for dynamic inventory and configuration.
- ✓ Available under ansible_facts.interfaces.*

EXAMPLES

- ansible_facts.default_ipv4.address
- ansible_facts.all_ipv4_addresses
- ansible_facts.interfaces
- ansible_facts.default_ipv4.gateway

5 OS FACTS



- ✓ Operating system, distribution, version, kernel, etc.
- ✓ Used for conditional logic and compatibility checks.
- ✓ Available under ansible_facts.os_family, etc.

EXAMPLES

- ansible_facts.os_family (RedHat, Debian, etc.)
- ansible_facts.distribution
- ansible_facts.distribution_version
- ansible_facts.kernel

6 CUSTOM FACTS



- ✓ You can define your own facts.
- ✓ Stored in /etc/ansible/facts.d/ on the node.
- ✓ Must be JSON or INI format.

EXAMPLE

```
JSON FACT (myapp.fact)
{
  "app_name": "MyApp",
  "version": "2.5.1",
  "env": "production"
}
```

```
INI FACT (myapp.fact)
[myapp]
app_name=MyApp
version=2.5.1
env=production
```

7 PERFORMANCE CONSIDERATIONS



- ✓ Gathering facts adds overhead, especially at scale.
- ✓ Use gather_subset to collect only what you need.
- ✓ Disable fact gathering for simple, stateless tasks.
- ✓ Cache facts to avoid re-gathering on every run.
- ✓ Use fact caching (JSON, Redis, Memcached, etc.) in ansible.cfg.

GOOD PRACTICE

- Use gather_subset: min for minimal facts.
- Use gather_subset: network for network automation.
- Use caching for large inventories.
- Refresh facts only when needed.

CONFIG EXAMPLE (ansible.cfg)

```
[defaults]
gathering = smart
fact_caching = jsonfile
fact_caching_connection = ~/.ansible/facts
fact_caching_timeout = 86400
```

QUICK EXAMPLE: USING FACTS IN PLAYBOOK

```
- name: Show OS and IP
  hosts: webservers
  tasks:
    - name: Display OS
      debug:
        msg: "OS: {{ ansible_facts.os_family }} {{ ansible_facts.distribution_version }}"
    - name: Display IP
      debug:
        msg: "IP: {{ ansible_facts.default_ipv4.address }}"
```

COMMON FACT PATHS

All facts (default path):
/etc/ansible/facts.d/
Windows custom facts:
C:\Program Files\ansible\facts.d\
Ansible built-in facts:
Available under ansible_facts.*



TIP

Facts empower your playbooks to adapt, decide, and automate intelligently.

★ PRO TIP: Facts turn data into decisions. Gather smart, cache wisely, and use intentionally. - VERIQTA



VERIQTA

➤ VERIQTA ⚡

VERIQTA



CONDITIONALS

MAKE SMART DECISIONS IN YOUR PLAYBOOKS

VERIQTA

“Conditionals bring intelligence to automation.
Right task. Right time. Right result.

— VERIQTA ”

WHY CONDITIONALS?

Conditionals help you run tasks only when needed. They make playbooks smarter, faster and production-ready.

1 WHEN STATEMENTS



- ✓ when controls whether a task runs.
- ✓ Evaluated before the task executes.
- ✓ Can use facts, variables, or expressions.
- ✓ Supports complex expressions.

EXAMPLE

```
- name: Install Nginx on RedHat family
  yum:
    name: nginx
    state: present
    when: ansible_os_family == "RedHat"
```

2 BOOLEAN LOGIC



- ✓ Use standard boolean operators.
- ✓ and, or, not
- ✓ Use parentheses for clarity.
- ✓ Case insensitive (True/False/true/false).

OPERATOR	MEANING
and	Both conditions must be true
or	At least one condition must be true
not	Inverts the result
==, !=, >, <, >=, <=	Comparison operators

3 MULTIPLE CONDITIONS

- ✓ Combine multiple conditions.
- ✓ Use parentheses to control precedence.
- ✓ Long expressions can span multiple lines.

EXAMPLE

```
- name: Run task for Ubuntu 20 or 22 with 4GB+ RAM
  debug:
    msg: "Running on supported Ubuntu with enough RAM"
  when: (ansible_distribution == "Ubuntu") and
        (ansible_distribution_version is version('20.04', '>=') or
         ansible_distribution_version is version('22.04', '>=') ) and
        (ansible_memtotal_mb | int >= 4096)
```

4 FACT-BASED EXECUTION



- ✓ Use gathered facts to drive decisions.
- ✓ Examples: OS, IP, memory, disk, interfaces.
- ✓ Makes playbooks dynamic and environment-aware.

EXAMPLE

```
- name: Install package based on OS
  package:
    name: httpd
    state: present
    when: ansible_os_family == "RedHat"
```

COMMON FACTS

- ansible_os_family
- ansible_distribution
- ansible_distribution_version
- ansible_architecture
- ansible_memtotal_mb
- ansible_facts['devices']
- ansible_default_ipv4.address

5 REGISTERED OUTPUT



- ✓ Register task output for later use.
- ✓ Use the registered variable in conditions.
- ✓ Great for command results, APIs, checks.

EXAMPLE

```
- name: Check if file exists
  command: test -f /etc/myapp.conf
  register: file_check
  ignore_errors: yes

- name: Load config if file exists
  include_tasks: load_config.yml
  when: file_check.rc == 0
```

RESULT FIELDS

- .rc (return code)
- .stdout (output)
- .stderr (error)
- .changed
- .failed

6 ERROR HANDLING



- ✓ Use conditionals with error handling.
- ✓ Ignore errors when expected.
- ✓ Fail gracefully with meaningful messages.

IGNORE ERRORS

```
- name: Try optional command
  command: /usr/bin/somecmd
  register: cmd_out
  ignore_errors: yes
  when: ansible_os_family == "Debian"
```

FAIL WITH MESSAGE

```
- name: Abort if unsupported OS
  fail:
    msg: "Unsupported OS!"
  when: ansible_os_family not in
        ['RedHat', 'Debian', 'Ubuntu']
```

7 PRODUCTION EXAMPLES



INSTALL BASED ON OS

```
when: ansible_os_family == "Debian"
apt:
  name: nginx
  state: present
```

RUN ONLY ON MASTER

```
when: inventory_hostname ==
      groups['masters'][0]
debug:
  msg: "Running on master"
```

SKIP IF PACKAGE EXISTS

```
when: nginx_status.rc != 0
command: systemctl start nginx
register: nginx_status
ignore_errors: yes
```

DISK SPACE CHECK

```
when: ansible_facts['devices']
      ['vda']['size'] | int > 10737418240
debug:
  msg: "Disk is larger than 10GB"
```

8 BEST PRACTICES



- ✓ Keep conditions simple and readable.
- ✓ Use parentheses for complex logic.
- ✓ Rely on facts instead of hardcoding.
- ✓ Validate assumptions before acting.
- ✓ Use meaningful variable names.
- ✓ Handle errors intentionally.
- ✓ Test across environments.
- ✓ Document your logic.



AVOID THESE MISTAKES

- ✗ Overly complex one-liners
- ✗ Hardcoded values
- ✗ Ignoring undefined variables
- ✗ Skipping error handling

★ PRO TIP: Conditionals turn playbooks into smart automation. Test. Validate. Trust.

— VERIQTA



YouTube
VERIQTA



GitHub
veriqta



LinkedIn
VERIQTA



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



LOOPS

AUTOMATE AT SCALE

VERIQTA

“ Loops allow Ansible to repeat tasks intelligently over data, lists, and complex structures.

– VERIQTA

WHY LOOPS?

- ✓ Eliminate repetition.
- ✓ Process lists and data.
- ✓ Build powerful automation.
- ✓ Scale your playbooks.
- ✓ Save time and reduce errors.

1 LOOP



- ✓ Modern loop keyword (recommended).
- ✓ Works with any module.
- ✓ Simple, readable, and powerful.
- ✓ Preferred over deprecated loops.

EXAMPLE

```
- name: Install packages
  ansible.builtin.yum:
    name: "{{ item }}"
    state: present
  loop:
    - nginx
    - git
    - curl
```

LOOP OUTPUT (item)

```
item = nginx
item = git
item = curl
```

2 WITH_ITEMS (LEGACY)



- ✓ Older way to loop over lists.
- ✓ Still seen in legacy playbooks.
- ✓ Use loop for new automation.

EXAMPLE

```
- name: Create users
  ansible.builtin.user:
    name: "{{ item }}"
    state: present
  with_items:
    - alice
    - bob
    - charlie
```

NOTE

with_items is deprecated. Use loop instead.

3 NESTED LOOPS



- ✓ Loop inside loop for multi-dimensional data.
- ✓ Useful for combinations and matrices.
- ✓ Handle with care for performance.

EXAMPLE: NESTED LOOPS

```
- name: Create users in multiple groups
  ansible.builtin.user:
    name: "{{ item.0 }}"
    groups: "{{ item.1 }}"
    state: present
  loop: "{{ users | product(groups) | list }}"
  vars:
    users: [alice, bob]
    groups: [dev_ops, qa]
```

RESULT

```
alice-dev
alice-ops
alice-qa
bob-dev
bob-ops
bob-qa
```

4 DICTIONARIES



- ✓ Loop over dictionary keys or items.
- ✓ Access key, value, or both.
- ✓ Use dict2items filter for full control.

EXAMPLE: KEYS

```
- name: Show dictionary keys
  ansible.builtin.debug:
    var: item.key
  loop: "{{ mydict | dict2items }}"
  vars:
    mydict:
      app: nginx
      port: 80
      env: production
```

EXAMPLE: KEYS & VALUES

```
- name: Show keys and values
  ansible.builtin.debug:
    msg: "{{ item.key }} = {{ item.value }}"
  loop: "{{ mydict | dict2items }}"
  vars:
    mydict:
      app: nginx
      port: 80
      env: production
```

5 RETRY LOOPS

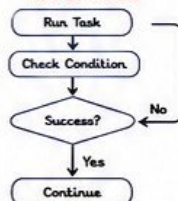


- ✓ Retry a task until success or timeout.
- ✓ Use until, retries, and delay.
- ✓ Essential for unstable operations.

EXAMPLE: RETRY UNTIL SERVICE IS READY

```
- name: Wait for web service
  ansible.builtin.uri:
    url: http://localhost:8080/health
    status_code: 200
  register: result
  until: result.status == 200
  retries: 10
  delay: 5
  ignore_errors: yes
```

HOW IT WORKS



6 COMPLEX ITERATIONS



- ✓ Use filters, maps, selects, and zips.
- ✓ Process complex data structures.
- ✓ Build powerful dynamic loops.

EXAMPLE: COMPLEX ITERATION

```
- name: Create users with attributes
  ansible.builtin.user:
    name: "{{ item.name }}"
    shell: "{{ item.shell }}"
    groups: "{{ item.groups | join(', ') }}"
    state: present
  loop: "{{ users | selectattr('active', 'equalto', true) | map(attribute='data') | list }}"
  vars:
    users:
      - name: alice data: { shell: /bin/bash, groups: ['dev'], active: true }
      - name: bob data: { shell: /bin/zsh, groups: ['ops'], active: false }
      - name: charlie data: { shell: /bin/bash, groups: ['qa', 'dev'], active: true }
```

7 ENTERPRISE AUTOMATION



- ✓ Loop over inventory, files, APIs, or cloud resources.
- ✓ Drive multi-environment automation.
- ✓ Make playbooks reusable and scalable.

EXAMPLE: DEPLOY TO MULTIPLE ENVIRONMENTS

```
- name: Deploy application
  ansible.builtin.include_role:
    name: app_deploy
  loop:
    - env: dev host_group: dev_servers version: "1.2.0"
    - env: stage host_group: stage_servers version: "1.2.0"
    - env: prod host_group: prod_servers version: "1.2.0"
  loop_control:
    loop_var: deploy
  vars:
    ansible_group: "{{ deploy.host_group }}"
    app_version: "{{ deploy.version }}"
    environment: "{{ deploy.env }}"
```

USE CASES

- Rolling updates
- Multi-region deployments
- User provisioning
- Cloud resource management
- Compliance at scale
- Data-driven automation



BEST PRACTICES

- ✓ Prefer loop over with_items.
- ✓ Use meaningful loop_var names.
- ✓ Keep loops simple and readable.
- ✓ Validate data before looping.
- ✓ Use when with loops for filtering.
- ✓ Avoid very large loops in a single task.
- ✓ Use retries for unstable operations.



COMMON MISTAKES

- ✗ Using with_items in new playbooks.
- ✗ Forgetting loop_var in nested loops.
- ✗ Looping over undefined or empty data.
- ✗ Complex one-liners that are hard to read.
- ✗ Not handling errors in loops.

☆ PRO TIP: Loops turn data into action. The better your data, the smarter your automation.

– VERIQTA



HANDLERS

RUN ONLY WHEN NEEDED

“ Handlers allow you to react to changes in your system in a controlled and efficient way.

— VERIQTA ”

WHY HANDLERS?

- ✓ Avoid unnecessary restarts.
- ✓ Run only when changes occur.
- ✓ Keep services stable.
- ✓ Improve performance and reliability.

1 NOTIFY



- ✓ Tasks notify handlers when changes occur.
- ✓ Use "notify" to trigger a handler.
- ✓ Handlers run once at the end of a play.
- ✓ Multiple tasks can notify the same handler.

EXAMPLE

```
- name: Update Nginx configuration
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
  notify: Restart Nginx
```

2 HANDLER EXECUTION



- ✓ Handlers run after all tasks in the play complete.
- ✓ Run only if notified by a task that changed something.
- ✓ Run once per play, even if notified multiple times.
- ✓ Can be flushed early if needed (meta: flush_handlers).

EXAMPLE PLAY FLOW

1. Tasks run
2. Tasks notify handlers (if changed)
3. Handlers run (once each)
4. Play ends

3 MULTIPLE NOTIFICATIONS



- ✓ Many tasks can notify the same handler.
- ✓ Handler still runs only once.
- ✓ Useful for shared services like Nginx, Apache, etc.

EXAMPLE

```
- name: Copy config file A
  ansible.builtin.copy:
    src: a.conf
    dest: /etc/app/a.conf
  notify: Restart App

- name: Copy config file B
  ansible.builtin.copy:
    src: b.conf
    dest: /etc/app/b.conf
  notify: Restart App
```

RESULT

Restart App handler runs only once, even though multiple tasks notified it.

4 RESTART SERVICES



- ✓ Most common use: restart or reload services.
- ✓ Other actions: reload configs, reindex, recompile, etc.
- ✓ Prevents unnecessary downtime and interruptions.

EXAMPLE HANDLERS

```
handlers:
- name: Restart Nginx
  ansible.builtin.service:
    name: nginx
    state: restarted

- name: Reload Systemd
  ansible.builtin.systemd:
    daemon_reload: yes

- name: Reload Application
  ansible.builtin.command: /usr/bin/appctl reload
```

5 ORDERING

1. Handlers run in the order they appear in the handlers section.
2. Use "listen" to group multiple handlers under one name.
3. Helps control execution order and reduce duplication.

EXAMPLE: ORDER MATTERS

```
handlers:
- name: Reload Nginx
  ansible.builtin.service:
    name: nginx
    state: reloaded

- name: Restart Nginx
  ansible.builtin.service:
    name: nginx
    state: restarted
```

EXAMPLE: USING LISTEN

```
handlers:
- name: Reload Nginx
  ansible.builtin.service:
    name: nginx
    state: reloaded
    listen: nginx

- name: Restart Nginx
  ansible.builtin.service:
    name: nginx
    state: restarted
    listen: nginx
```

6 BEST PRACTICES



- ✓ Keep handlers simple and focused.
- ✓ Use meaningful handler names.
- ✓ Group related handlers with "listen".
- ✓ Avoid long-running tasks in handlers.
- ✓ Test handlers independently.

BEST PRACTICES SUMMARY

- Use handlers for changes that require actions.
- Avoid unconditional restarts.
- Use listen for better organization.
- Keep handlers idempotent and fast.
- Always test before deploying to production.

7 PRODUCTION WORKFLOWS



CONFIG CHANGE

A task changes a file or configuration.



NOTIFY HANDLER

The task notifies the appropriate handler.



COMPLETE PLAY

All tasks in the play are executed.



RUN HANDLERS

Handlers run once each at the end.



SYSTEM STABLE

Services running with updated configuration.

FULL EXAMPLE

```
- name: Deploy web application
  hosts: webservers
  become: yes
  tasks:
    - name: Deploy config
      ansible.builtin.template:
        src: app.conf.j2
        dest: /etc/app/app.conf
      notify: Restart App

    - name: Deploy code
      ansible.builtin.copy:
        src: app/
        dest: /var/www/app/
      notify: Restart App
```

HANDLERS

```
handlers:
- name: Restart App
  ansible.builtin.service:
    name: app
    state: restarted
```

BEHAVIOR

- If any task changes something and notifies Restart App, the service restarts.
- If no changes occur, the handler does not run.



TIP

Handlers are your safety net. They act only when action is truly required.



PRO TIP: Use handlers to make automation efficient, safe, and predictable.

— VERIQTA



VERIQTA

TEMPLATES

DYNAMIC CONFIGURATION WITH JINJA2



WHY TEMPLATES?

- ✓ Eliminate hardcoded values.
- ✓ Generate dynamic configs.
- ✓ Adapt to environments.
- ✓ Ensure consistency.
- ✓ Scale configuration management.

“ Templates allow you to generate files dynamically using variables, logic, and loops. Jinja2 is the templating engine used by Ansible. ”

- VERIQTA

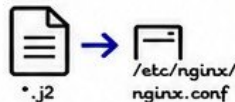
1 TEMPLATES OVERVIEW



- ✓ Templates are text files with Jinja2 syntax.
- ✓ Rendered on the control node.
- ✓ Result is transferred to the managed node.
- ✓ Use the template module.

EXAMPLE: TEMPLATE MODULE

```
- name: Deploy Nginx configuration from template
  ansible.builtin.template:
    src: nginx.conf.j2
    dest: /etc/nginx/nginx.conf
    owner: root
    group: root
    mode: '0644'
```



2 JINJA2 VARIABLES



- ✓ Use variables defined in inventory, playbooks, or facts.
- ✓ Access variables using double curly braces.
- ✓ Supports dot notation and dictionary access.

EXAMPLE

```
server {
  listen {{ nginx_port }};
  server_name {{ server_name }};
  root {{ web_root }};
}
```

VARIABLE SOURCES

- Inventory variables
- Group / host vars
- Playbook vars
- Facts
- Extra vars (-e)
- Registered vars

3 FILTERS



- ✓ Filters modify and format data.
- ✓ Improve readability and flexibility.
- ✓ Use the pipe symbol (|) to apply filters.

COMMON FILTER EXAMPLES

{{ variable default('value') }}	Set default value if undefined
{{ variable upper }}	Convert to UPPERCASE
{{ variable lower }}	Convert to lowercase
{{ variable int }}	Convert to integer
{{ variable join(',') }}	Join list items
{{ variable to_nice_yaml }}	Pretty YAML output
{{ variable regex_replace('a','b') }}	Replace using regex
{{ variable trim }}	Remove leading/trailing spaces

4 CONDITIONALS



- ✓ Render content based on conditions.
- ✓ Use if, elif, else, and endif.
- ✓ Great for environment specific configs.

EXAMPLE

```
{% if ansible_os_family == 'RedHat' %}
package: httpd
service: httpd
{% elif ansible_os_family == 'Debian' %}
package: nginx
service: nginx
{% else %}
# Default configuration
{% endif %}
```

USE CASES

- OS specific configs
- Environment toggles
- Feature flags
- Conditional params

5 LOOPS



- ✓ Loop over lists, dictionaries, or ranges.
- ✓ Use for loops in templates.
- ✓ Perfect for repetitive configuration blocks.

EXAMPLE: LOOP OVER LIST

```
{% for server in app_servers %}
server {{ loop.index }}: {{ server.ip }}:{{ server.port }}
{% endfor %}
```

COMMON LOOP VARIABLES

- loop.index
- loop.index0
- loop.first
- loop.last
- loop.length
- loop.revindex

EXAMPLE: LOOP OVER DICTIONARY

```
{% for key, value in app_config.items() %}
{{ key }} = {{ value }}
{% endfor %}
```

6 CONFIGURATION GENERATION



- ✓ Generate complex files dynamically.
- ✓ Reduce duplication and manual edits.
- ✓ Ensure consistent, environment-aware configs.

EXAMPLE: GENERATE NGINX UPSTREAM

```
upstream backend_servers {
  {% for backend in backends %}
  server {{ backend.ip }}:{{ backend.port }};
  {% endfor %}
}
```

USE CASES

- Web server configs
- App configs
- Network configs
- Monitoring configs
- Cloud configs

7 ENTERPRISE TEMPLATING



- ✓ Use template inheritance with includes.
- ✓ Store templates in version control.
- ✓ Standardize templates across teams.
- ✓ Leverage roles for reusable templates.

BEST PRACTICES

- Keep templates simple and modular.
- Use meaningful variable names.
- Validate output before deployment.
- Document variables and dependencies.
- Test templates with different inputs.

ENTERPRISE STRATEGIES

- Separate templates by role.
- Use default values.
- Implement environment overlays.
- Enforce code review.
- Automate template testing in CI/CD.

8 COMMON MISTAKES



- ✗ Hardcoding values instead of using variables.
- ✗ Complex logic in templates (keep it simple).
- ✗ Not validating generated configurations.
- ✗ Ignoring whitespace and indentation.
- ✗ Overusing conditionals and nested loops.



TROUBLESHOOTING TIPS

- Use ansible-playbook --syntax-check.
- Use --check --diff to preview changes.
- Debug with {{ variable }} and {{ variable | to_nice_yaml }}.
- Validate rendered files on target systems.
- Use -vvv for detailed output.

PRODUCTION CHECKLIST

- ✓ Template version controlled
- ✓ Variables documented
- ✓ Validated before deployment
- ✓ Tested in all environments
- ✓ Monitored after deployment

TEMPLATE WORKFLOW



★ PRO TIP: Templates turn variables into powerful, dynamic configurations. Keep them clean, test them always!

- VERIQTA



ANSIBLE

VERIQTA

MANAGING FILES

CONTROL, DEPLOY & AUTOMATE FILES WITH ANSIBLE

“ Ansible provides powerful modules to manage files and directories consistently across your infrastructure. ”
- VERIQTA

WHY MANAGE FILES?

- ✓ Ensure consistency.
- ✓ Automate configuration.
- ✓ Reduce manual errors.
- ✓ Maintain security.
- ✓ Improve reliability.

1 COPY MODULE



- ✓ Copy files from control node to managed nodes.
- ✓ Supports attributes, backup, and permissions.
- ✓ Best for static files.

EXAMPLE

```
- name: Copy application config
  ansible.builtin.copy:
    src: files/app.conf
    dest: /etc/app/app.conf
    owner: appuser
    group: appgroup
    mode: '0644'
    backup: yes
```

USE CASES

- Static configuration files
- Certificates
- Scripts
- Binary files

2 TEMPLATE MODULE



- ✓ Deploy templates with Jinja2 variables.
- ✓ Dynamic content at deployment time.
- ✓ Great for configs and structured files.

EXAMPLE

```
- name: Deploy Nginx config from template
  ansible.builtin.template:
    src: templates/nginx.conf.j2
    dest: /etc/nginx/nginx.conf
    owner: root
    group: root
    mode: '0644'
    backup: yes
```

USE CASES

- Application configs
- Service configs
- Dynamic files
- Environment specific configs

3 FILE MODULE



- ✓ Manage file or directory attributes.
- ✓ Create, delete, or modify permissions.
- ✓ Does not handle content.

EXAMPLE

```
- name: Ensure directory exists
  ansible.builtin.file:
    path: /var/log/myapp
    state: directory
    owner: appuser
    group: appgroup
    mode: '0755'
```

COMMON STATES

- file - ensure file exists/absent
- directory - ensure directory
- link - create symbolic link
- absent - remove file/dir
- touch - update file timestamp

4 LINEinfile MODULE



- ✓ Ensure a single line is present or absent.
- ✓ Useful for simple key/value updates.
- ✓ Idempotent line management.

EXAMPLE

```
- name: Set MaxOpenFiles in limits.conf
  ansible.builtin.lineinfile:
    path: /etc/security/limits.conf
    line: '* soft nofile 65535'
    create: yes
    backup: yes
```

USE CASES

- Update single values
- Add settings
- Comment/uncomment lines

5 BLOCKinfile MODULE



- ✓ Insert, update, or remove a block of text.
- ✓ Marks block with custom markers.
- ✓ Ideal for multi-line configurations.

EXAMPLE

```
- name: Manage custom sysctl block
  ansible.builtin.blockinfile:
    path: /etc/sysctl.conf
    marker: '# (mark) ANSIBLE MANAGED BLOCK'
    block: |
      net.ipv4.ip_forward = 1
      net.ipv4.tcp_syncookies = 1
    create: yes
    backup: yes
```

USE CASES

- Multi-line configs
- Firewall rules
- Feature toggles
- System tuning

6 SYNCHRONIZE MODULE



- ✓ Efficiently synchronize files/directories (rsync).
- ✓ Delta transfers save bandwidth.
- ✓ Great for large directories.

EXAMPLE

```
- name: Sync web assets
  ansible.builtin.synchronize:
    src: /var/www/assets/
    dest: /var/www/assets/
    delete: yes
    archive: yes
    compress: yes
```

USE CASES

- Website assets
- Logs & backups
- Static files
- Large directories

7 PRODUCTION RECOMMENDATIONS



SECURITY FIRST

- Set correct permissions.
- Use least privilege.
- Avoid 777 permissions.
- Protect sensitive files.



ALWAYS BACKUP

- Use backup: yes before modifying files.
- Store backups securely.



IDEMPOTENT TASKS

- Write tasks that can run multiple times without changing state unnecessarily.



USE TEMPLATES

- Prefer template over copy for any file that changes per environment.



HIGH PERFORMANCE

- Use synchronize for large data.
- Avoid unnecessary file transfers.

★ PRO TIP: Choose the right module for the job. Consistent file management is the foundation of reliable infrastructure. - VERIQTA



MANAGING PACKAGES

INSTALL, UPDATE & CONTROL SOFTWARE WITH ANSIBLE

“ Ansible makes package management consistent, idempotent, and reliable across Linux and macOS environments.

— VERIQTA ”

WHY MANAGE PACKAGES?

- ✓ Ensure consistency.
- ✓ Maintain security.
- ✓ Automate deployments.
- ✓ Control versions.
- ✓ Reduce configuration drift.
- ✓ Improve reliability.

1 APT (DEBIAN / UBUNTU)



- ✓ Manages packages on Debian-based systems.
- ✓ Uses apt package manager.
- ✓ Supports install, update, remove, upgrade.
- ✓ Works with repositories and keys.

EXAMPLE

```
- name: Install nginx
  ansible.builtin.apt:
    name: nginx
    state: present
    update_cache: yes
    cache_valid_time: 3600
```

USE CASES

- Web servers
- Application deployments
- Security updates
- System maintenance

2 DNF (FEDORA / RHEL 8+)



- ✓ Manages packages on modern RHEL, Fedora, CentOS Stream.
- ✓ Faster than yum.
- ✓ Handles dependencies automatically.
- ✓ Supports module streams.

EXAMPLE

```
- name: Install httpd
  ansible.builtin.dnf:
    name: httpd
    state: present
    update_cache: yes
```

USE CASES

- RHEL 8+ systems
- Web servers
- Enterprise applications
- System updates

3 YUM (RHEL 7 / CENTOS 7)



- ✓ Legacy package manager for RHEL/CentOS 7.
- ✓ Handles install, update, remove.
- ✓ Uses repositories and dependencies.
- ✓ Still widely used in production.

EXAMPLE

```
- name: Install nginx
  ansible.builtin.yum:
    name: nginx
    state: present
    update_cache: yes
```

USE CASES

- RHEL 7 systems
- Legacy environments
- Web servers
- Maintenance

4 PACKAGE MODULE (UNIFIED)



- ✓ Abstracts package management across platforms.
- ✓ Automatically selects the right package manager.
- ✓ Great for cross-distribution playbooks.
- ✓ Use when targeting multiple OS families.

EXAMPLE

```
- name: Install vim (auto-detect manager)
  ansible.builtin.package:
    name: vim
    state: present
```

USE CASES

- Cross-platform playbooks
- Mixed OS environments
- Simplified roles

5 HOMEBREW (MACOS)



- ✓ Manages packages on macOS.
- ✓ Installs CLI tools, apps, and libraries.
- ✓ Uses brew package manager.
- ✓ Ideal for dev machines and CI runners.

EXAMPLE

```
- name: Install git with Homebrew
  community.general.homebrew:
    name: git
    state: present
```

USE CASES

- macOS development
- Dev tools installation
- CI/CD runners
- Local automation

6 VERSION PINNING



- ✓ Control exact package versions.
- ✓ Prevent unexpected updates.
- ✓ Ensure stability and repeatability.
- ✓ Use version or epoch:version-release.

EXAMPLE

```
- name: Install nginx specific version
  ansible.builtin.apt:
    name: nginx=1.18.0-6ubuntu14.4
    state: present
```

USE CASES

- Production stability
- Compliance requirements
- Reproducible builds
- Rollback control

7 REPOSITORY MANAGEMENT



- ✓ Manage repositories for software sources.
- ✓ Add, update, enable or disable repos.
- ✓ Import GPG keys securely.
- ✓ Ensure packages come from trusted sources.

EXAMPLE (APT REPOSITORY)

```
- name: Add nginx repo
  ansible.builtin.apt_repository:
    repo: "deb http://nginx.org/packages/ubuntu focal nginx"
    state: present
    filename: nginx
    update_cache: yes
```

USE CASES

- Third-party software
- Internal repositories
- Updates management
- Security compliance

COMMON PACKAGE STATES

STATE	DESCRIPTION
present	Ensure package is installed.
absent	Ensure package is removed.
latest	Ensure package is at the latest version.
installed	Alias for present.
removed	Alias for absent.
held	Prevent package from being updated.

PRODUCTION RECOMMENDATIONS



Always update cache
Update package cache
before installing.
Improves reliability.



Pin critical packages
Pin versions for critical
components to avoid
unexpected changes.



Use repositories
Use official or trusted
repositories only.
Avoid unknown sources.



Test updates
Test package updates in
staging before rolling
out to production.



Monitor & audit
Monitor package versions
regularly and audit
for vulnerabilities.

★ PRO TIP: Use the package module for portability and version pinning for stability. Automate and sleep peacefully.

— VERIQTA



ANSIBLE

VERIQTA



MANAGING SERVICES

CONTROL, MONITOR & AUTOMATE SERVICES WITH ANSIBLE

“ Ansible provides powerful modules to manage services using systemd or SysV init systems consistently across environments.

– VERIQTA ”

WHY MANAGE SERVICES?

- ✓ Ensure high availability.
- ✓ Automate starts, stops & restarts.
- ✓ Enforce configuration changes.
- ✓ Maintain security and reliability.
- ✓ Validate service health.

1 SYSTEMD OVERVIEW



- ✓ systemd is the init system for modern Linux.
- ✓ Manages services, sockets, timers, and targets.
- ✓ Provides parallel startup and better reliability.
- ✓ Required for most enterprise distributions.

COMMON SYSTEMCTL COMMANDS

```
systemctl start <service>    # Start service
systemctl stop <service>     # Stop service
systemctl restart <service>  # Restart service
systemctl status <service>   # Check status
systemctl enable <service>   # Enable on boot
systemctl disable <service>  # Disable on boot
```

2 SERVICE MODULE BASICS



- ✓ Manage services on both systemd and SysV systems.
- ✓ Supports start, stop, restart, reload, and more.
- ✓ Handles enabled/disabled states.
- ✓ Idempotent and reliable.

EXAMPLE

```
- name: Ensure nginx is running
  ansible.builtin.service:
    name: nginx
    state: started
    enabled: yes
```

COMMON PARAMETERS

- name: service name
- state: started, stopped, restarted, reloaded
- enabled: yes or no
- daemon_reload: yes or no
- arguments: extra args

3 ENABLE SERVICES



- ✓ Ensure services start on boot.
- ✓ Prevent manual configuration drift.
- ✓ Use enabled: yes for persistence.

EXAMPLE

```
- name: Enable and start sshd
  ansible.builtin.service:
    name: sshd
    state: started
    enabled: yes
```

USE CASES

- Critical system services
- Security services
- Monitoring agents
- Database services

4 RESTART POLICIES



- ✓ Control how services behave after failures.
- ✓ Configure systemd restart policies.
- ✓ Improve resilience and uptime.

SYSTEMD RESTART POLICIES

```
no          - Do not restart
on-failure  - Restart on failure
on-abnormal - Restart on abnormal exit
on-watchdog - Restart on watchdog
always      - Always restart
```

EXAMPLE (DROP-IN CONFIG)

```
[Service]
Restart=on-failure
RestartSec=5s
StartLimitIntervalSec=0
StartLimitBurst=3
```

5 VALIDATION



- ✓ Validate service status after changes.
- ✓ Fail fast if service is not running.
- ✓ Use handlers and checks for safety.

EXAMPLE

```
- name: Restart nginx
  ansible.builtin.service:
    name: nginx
    state: restarted
  register: nginx_result
  notify: Validate Nginx
```

REGISTERED OUTPUT EXAMPLE

```
nginx_result:
  changed: true
  status:
    ActiveState: active
    SubState: running
```

6 HEALTH VERIFICATION



- ✓ Verify service health beyond just status.
- ✓ Check ports, endpoints, and processes.
- ✓ Use uri, wait_for, and command modules.

EXAMPLE: CHECK HTTP SERVICE

```
- name: Verify web service is healthy
  ansible.builtin.uri:
    url: http://localhost:80/health
    status_code: 200
    return_content: no
  register: health_check
```

OTHER HEALTH CHECKS

- wait_for: port check
- command: process check
- shell: custom scripts
- uri: HTTP/HTTPS check

7 PRODUCTION AUTOMATION WORKFLOWS



1. DEPLOY

- Copy config
- Install packages



2. CONFIGURE

- Update settings
- Template configs



3. MANAGE SERVICES

- Start / Enable
- Restart if changed



4. VALIDATE

- Check status
- Verify health



5. NOTIFY & LOG

- Alert on failure
- Log results

PRODUCTION RECOMMENDATIONS



IDEMPOTENT & SAFE

- Write idempotent playbooks.
- Use handlers to reduce restarts.



MINIMIZE DOWNTIME

- Reload instead of restart.
- Use rolling updates.



MONITOR & ALERT

- Monitor service health.
- Alert on failures.



DOCUMENT & STANDARDIZE

- Document service changes.
- Follow naming standards.



TEST & AUTOMATE

- Test in staging first.
- Automate everything.

★ PRO TIP: Manage services reliably. Validate health. Automate recovery. Keep systems running.

– VERIQTA



YouTube
VERIQTA



GitHub
veriqta



LinkedIn
VERIQTA



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



ANSIBLE

VERIQTA

VERIQTA

ROLES

MODULAR, REUSABLE & SCALABLE AUTOMATION
WITH ANSIBLE



WHY USE ROLES?

- ✓ Reusable and maintainable.
- ✓ Standardized structure.
- ✓ Encourages best practices.
- ✓ Easy to share across teams.
- ✓ Scales automation efficiently.

“ Roles help you organize playbooks, reuse code, enforce standards, and scale automation across teams and environments. ”

- VERIQTA

1 ROLE STRUCTURE OVERVIEW



- ✓ Roles have a standard directory layout.
- ✓ Each directory has a specific purpose.
- ✓ Keeps projects clean and maintainable.
- ✓ Promotes reusability and consistency.

TYPICAL ROLE DIRECTORY

```
myrole/
├── defaults/      # Default variables
├── vars/          # Role-specific variables
├── tasks/         # Main tasks
├── handlers/      # Handlers
├── templates/     # Jinja2 templates
├── files/         # Static files
├── meta/          # Role metadata
├── tests/         # Test scenarios (optional)
└── README.md      # Role documentation
```

KEY BENEFITS

- Reusability
- Maintainability
- Standardization
- Collaboration
- Scalability

2 DEFAULTS



- ✓ Default variables with the lowest precedence.
- ✓ Can be overridden by inventory or extra vars.
- ✓ Used for safe, sensible default values.

EXAMPLE: defaults/main.yml

```
---
nginx_package: nginx
nginx_service_state: started
nginx_service_enabled: true
nginx_port: 80
```

BEST PRACTICES

- Use defaults for baseline values.
- Never use defaults for secrets.
- Document all default variables.

3 TASKS



- ✓ Main list of tasks executed by the role.
- ✓ Keep tasks small, focused, and idempotent.
- ✓ Include files for better organization.

EXAMPLE: tasks/main.yml

```
---
- name: Install nginx
  apt:
    name: "{{ nginx_package }}"
    state: present
- name: Start nginx
  service:
    name: nginx
    state: "{{ nginx_service_state }}"
    enabled: "{{ nginx_service_enabled }}"
```

BEST PRACTICES

- One task = one job.
- Use modules, not shell.
- Add meaningful names.
- Keep tasks idempotent.
- Use include_tasks for complex workflows.

4 HANDLERS



- ✓ Handlers run when notified by tasks.
- ✓ Used for actions like restarts or reloads.
- ✓ Run only once per play, even if notified multiple times.

EXAMPLE: handlers/main.yml

```
---
- name: Restart nginx
  service:
    name: nginx
    state: restarted
```

BEST PRACTICES

- Use handlers for restarts.
- Name handlers clearly.
- Keep handlers lightweight.
- Don't forget to notify.

5 TEMPLATES



- ✓ Jinja2 templates for dynamic configuration.
- ✓ Stored in templates/ directory.
- ✓ Use variables for flexibility.

EXAMPLE: templates/nginx.conf.j2

```
server {
    listen {{ nginx_port }};
    server_name {{ server_name }};
    root {{ doc_root }};
}
```

BEST PRACTICES

- Use meaningful variable names.
- Keep templates simple.
- Validate rendered output.
- Store only templates, not generated files.

6 FILES



- ✓ Static files copied as-is.
- ✓ Used for binaries, configs, certificates, scripts, etc.
- ✓ Stored in files/ directory.

EXAMPLE: files/

```
• myscript.sh
• app.conf
• ssl/cert.pem
• banner.txt
```

BEST PRACTICES

- Use for files that don't change.
- Set correct permissions.
- Avoid large files; use URLs if needed.

7 META



- ✓ Defines role metadata and dependencies.
- ✓ Supports galaxy.yml, author, license, etc.
- ✓ Enables role dependencies.

EXAMPLE: meta/main.yml

```
---
dependencies:
  - role: common
  - role: firewall
galaxy_info:
  author: Veriqta
  description: Nginx server role
  license: MIT
```

USE CASES

- Declare dependencies.
- Provide role information.
- Control role behavior.
- Enable Galaxy publishing.

8 ENTERPRISE ORGANIZATION



- ✓ Organize roles at scale.
- ✓ Use naming standards.
- ✓ Version control roles.
- ✓ Share via internal Galaxy or SCM.

```
project/
├── roles/
│   ├── common/
│   ├── nginx/
│   ├── database/
│   └── monitoring/
└── playbooks/
```

```
└── inventory/
    └── group_vars/
```

RECOMMENDED STRUCTURE

- One role = one responsibility.
- Keep roles independent.
- Use semantic versioning.
- Document roles (README).
- Test roles before reuse.
- Use CI/CD for roles.

PRODUCTION RECOMMENDATIONS



KEEP ROLES SMALL
Small roles are easier to test, reuse, and maintain.



USE ROLES EVERYWHERE
Avoid long playbooks. Promote role-based development.



STANDARDIZE
Structure & naming across teams for consistency.



DOCUMENT
Document variables, dependencies, and usage clearly.



TEST ROLES
Use Molecule or test playbooks to validate roles.



SECURITY FIRST
Avoid hardcoding secrets. Use Ansible Vault or external secret stores.

★ **PRO TIP:** Build roles like components. Reuse, version, test, and improve them continuously.

- VERIQTA



YouTube
VERIQTA



GitHub
veriqta



LinkedIn
VERIQTA



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



ANSIBLE

VERIQTA



COLLECTIONS

EXTEND, SHARE & STANDARDIZE AUTOMATION WITH ANSIBLE

“ Collections package roles, modules, plugins, playbooks, and more into a single, versioned unit for sharing, reuse, and enterprise automation at scale. ”

- VERIQTA

WHY USE COLLECTIONS?

- ✓ Pack and share reusable content.
- ✓ Standardize automation.
- ✓ Simplify distribution.
- ✓ Manage versions and dependencies.
- ✓ Promote best practices.

1 INSTALLING COLLECTIONS



- ✓ Install collections from Ansible Galaxy or local sources.
- ✓ Use ansible-galaxy collection install.
- ✓ Specify version requirements.
- ✓ Keep collections updated.

EXAMPLE COMMANDS

```
# Install from Galaxy
ansible-galaxy collection install
community.general
ansible-galaxy collection install
ansible.posix
# Install from requirements file
ansible-galaxy collection install
-r requirements.yml
# Install from local path
ansible-galaxy collection install
/path/to/collection.tar.gz
```

USE CASES

- Standardize modules across teams
- Deploy certified content
- Ensure compatible versions
- Offline environments

2 ANSIBLE GALAXY



- ✓ Central hub for discovering and sharing content.
- ✓ Search, review, and download collections.
- ✓ Verified and community content available.
- ✓ Supports namespaces and quality scoring.

EXAMPLE COMMANDS

```
# Search collections
ansible-galaxy collection search nginx
# View collection info
ansible-galaxy collection info
community.general
# List installed collections
ansible-galaxy collection list
```

BEST PRACTICES

- Use verified collections when possible.
- Check quality score and activity.
- Review documentation before use.
- Keep collections up to date.

3 CUSTOM COLLECTIONS



- ✓ Create internal collections for organization-specific automation.
- ✓ Package roles, modules, plugins, and docs.
- ✓ Publish to private Galaxy or distribute offline.
- ✓ Maintain organization standards.

EXAMPLE: CREATE COLLECTION

```
ansible-galaxy collection init mycompany.web
# Creates structure:
mycompany/web/
├── README.md
├── docs/
├── galaxy.yml
├── plugins/
│   ├── modules/
│   │   └── module_utils/
├── playbooks/
└── tests/
```

USE CASES

- Internal platform modules
- Company standards
- Reusable roles
- Compliance automation
- Share across teams

4 VERSION CONTROL



- ✓ Keep collections in Git repositories.
- ✓ Tag releases and manage versions.
- ✓ Use semantic versioning (SemVer).
- ✓ Review changes through pull requests.

EXAMPLE WORKFLOW

1. Create feature branch
2. Update collection content
3. Run tests
4. Create pull request
5. Review and merge
6. Tag release (e.g., v1.2.0)
7. Publish new version

BEST PRACTICES

- Follow GitFlow or trunk-based workflow.
- Tag releases clearly.
- Document changes in CHANGELOG.md.
- Protect main branch.

5 DEPENDENCIES



- ✓ Collections can depend on other collections.
- ✓ Define dependencies in galaxy.yml.
- ✓ Ansible Galaxy resolves dependencies.
- ✓ Use strict version constraints for stability.

EXAMPLE: galaxy.yml

```
namespace: mycompany
name: web
version: 1.2.0
dependencies:
  - name: community.general
    version: ">=8.0.0,<9.0.0"
  - name: ansible.posix
    version: ">=1.5.0"
```

BEST PRACTICES

- Pin compatible versions.
- Avoid overly broad constraints.
- Test with dependencies.
- Document requirements.

6 COLLECTION CONTENT



- ✓ Collections can include many content types.
- ✓ Organize content by purpose.
- ✓ Keep structure clean and consistent.
- ✓ Add documentation and tests.

CONTENT TYPES

- Modules (plugins/modules/)
- Roles (roles/)
- Plugins (lookup, filter, callback, etc.)
- Playbooks (playbooks/)
- Vars, defaults, files, templates
- Docs (docs/)
- Tests (tests/units/, tests/integration/)

STRUCTURE EXAMPLE

```
mycompany/web/
├── plugins/
│   ├── roles/
│   └── playbooks/
├── files/
├── templates/
├── docs/
└── tests/
```

7 ENTERPRISE REUSE



- ✓ Use collections to promote reuse across the organization.
- ✓ Reduce duplication and drift.
- ✓ Enforce standards and compliance.
- ✓ Empower teams with self-service automation.

ENTERPRISE BENEFITS

- Faster development
- Consistent automation
- Centralized governance
- Easier onboarding
- Better security and compliance

USE CASES

- Platform engineering teams
- Security and compliance
- Infrastructure provisioning
- Application deployments
- Operational automation

8 BEST PRACTICES SUMMARY



- ✓ Design collections for clarity and reuse.
- ✓ Follow naming and structure standards.
- ✓ Test thoroughly before publishing.
- ✓ Document everything.
- ✓ Version, release, and deprecate responsibly.

DO'S

- ✓ Use clear namespaces.
- ✓ Keep collections focused.
- ✓ Automate tests in CI/CD.
- ✓ Use semantic versioning.
- ✓ Provide examples and docs.

DON'TS

- ✗ Don't mix unrelated content.
- ✗ Don't ignore dependencies.
- ✗ Don't skip tests.
- ✗ Don't publish unreviewed code.
- ✗ Don't break backward compatibility.

PRODUCTION RECOMMENDATIONS



STANDARDIZE

Use collections for all reusable content.



SECURE

Verify sources and limit permissions.



TEST

Automate testing for reliability.



PUBLISH

Use private Galaxy or internal repo.

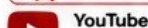


IMPROVE

Monitor usage and continuously improve.

★ PRO TIP: Collections turn good automation into great automation. Build once, share everywhere, scale forever.

- VERIQTA





ANSIBLE

➤ VERIQTA ⚡



ANSIBLE GALAXY

DISCOVER, SHARE & MANAGE AUTOMATION ON
CONTENT AT SCALE

“ Ansible Galaxy is the hub for discovering, sharing, and managing roles and collections. It enables reuse, consistency, and collaboration across the Ansible community and enterprises. ”

- VERIQTA

WHY USE ANSIBLE GALAXY?

- ✓ Reuse high-quality automation.
- ✓ Share and collaborate easily.
- ✓ Standardize across teams.
- ✓ Discover community content.
- ✓ Manage versions and dependencies.
- ✓ Promote best practices.

1 PUBLIC ROLES



- ✓ Thousands of roles available for free.
- ✓ Search, install, and use in playbooks.
- ✓ Community reviewed and maintained.
- ✓ Speeds up automation development.

SEARCH EXAMPLE

ansible-galaxy search nginx
ansible-galaxy search geerlingguy.nginx

INSTALL EXAMPLE

ansible-galaxy install geerlingguy.nginx
ansible-galaxy install username.role_name

2 COLLECTIONS



- ✓ Collections bundle roles, plugins, modules, and docs.
- ✓ Provide namespaces (e.g. community.general).
- ✓ Versioned, consistent, and reproducible.
- ✓ Recommended for modern Ansible usage.

LIST COLLECTIONS

ansible-galaxy collection list

INSTALL COLLECTION

ansible-galaxy collection install
community.general

3 PUBLISHING CONTENT



- ✓ Publish roles and collections for others to use.
- ✓ Requires a Galaxy account and API token.
- ✓ Follow naming and content guidelines.
- ✓ Quality score improves discoverability.

PUBLISH ROLE

ansible-galaxy role init myrole
Add content to role
ansible-galaxy role publish

PUBLISH COLLECTION

ansible-galaxy collection init my_ns.my_coll
Add content to collection
ansible-galaxy collection publish

4 VERSIONING



- ✓ Use Semantic Versioning (SemVer).
- ✓ Tracks features, fixes, and compatibility.
- ✓ Lock versions for stability.
- ✓ Avoid breaking changes in production.

VERSION EXAMPLE

1.2.0 → New Features
1.1.0 → Bug Fixes
1.0.0 → Initial Release

INSTALL SPECIFIC VERSION

ansible-galaxy collection install
community.general:3.8.0

5 ENTERPRISE REPOSITORIES



- ✓ Host content internally for security and compliance.
- ✓ Use Automation Hub or private Galaxy.
- ✓ Control access, review, and approvals.
- ✓ Ensure offline availability.

SETUP PRIVATE GALAXY SERVER

Using Automation Hub (recommended)
Install Automation Controller / Automation Hub
Configure Galaxy Content
Sync or import collections
Manage RBAC and approvals

BENEFITS

- Data stays internal
- Access control
- Audit and compliance
- Offline use
- Consistent standards

6 DEPENDENCY MANAGEMENT



- ✓ Collections can depend on other collections.
- ✓ Define dependencies in galaxy.yml.
- ✓ Ansible resolves and installs dependencies.
- ✓ Use version constraints for stability.

EXAMPLE: galaxy.yml

```
namespace: mycompany
name: myapp
version: 1.2.0
dependencies:
  - name: community.general
    version: ">=8.0.0, <9.0.0"
  - name: ansible.posix
    version: ">=1.5.0"
```

BEST PRACTICES

- Pin compatible versions
- Avoid overly broad ranges
- Test with dependencies
- Document requirements

7 COMMON GALAXY COMMANDS



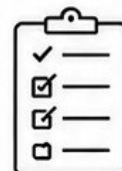
```
ansible-galaxy search <keyword>
ansible-galaxy install <role>
ansible-galaxy list
ansible-galaxy collection list
ansible-galaxy collection install <ns.coll>
ansible-galaxy collection info <ns.coll>
ansible-galaxy role init <rolename>
ansible-galaxy collection init <ns.coll>
ansible-galaxy role publish
ansible-galaxy collection publish
```

```
# Search roles
# Install a role
# List installed roles
# List installed collections
# Install a collection
# Show collection info
# Create new role
# Create new collection
# Publish role
# Publish collection
```

8 BEST PRACTICES



- ✓ Prefer collections over roles for new development.
- ✓ Use fully qualified collection names (FQCN).
- ✓ Pin versions in requirements for reproducibility.
- ✓ Regularly update and review dependencies.
- ✓ Validate content before promoting to production.
- ✓ Use private repositories for enterprise workloads.
- ✓ Follow naming, documentation, and quality guidelines.
- ✓ Automate testing and publishing pipelines.



GALAXY WORKFLOW OVERVIEW



DISCOVER

Search Galaxy
for content



INSTALL

Install roles or
collections



USE

Use in playbooks
and automation



CUSTOMIZE

Extend and adapt
for your needs



PUBLISH

Share back with
Galaxy



MAINTAIN

Update, version,
and improve

PRODUCTION RECOMMENDATIONS



TRUST & VERIFY
Use verified content.
Check quality score
and reviews.



SECURITY FIRST
Scan content for
vulnerabilities.
Use private repos.



STANDARDIZE
Enforce naming,
structure, and
documentation.



COLLABORATE
Share internally.
Review and approve
contributions.



CONTINUOUS IMPROVEMENT
Monitor usage, update
regularly, and retire
unused content.

★ **PRO TIP:** Use Galaxy to build on the community's work, but standardize and secure it for your enterprise.

- VERIQTA



YouTube
VERIQTA



GitHub
veriqta



LinkedIn
VERIQTA



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



ANSIBLE

➤ VERIQTA ⚡



SECRETS MANAGEMENT

SECURE YOUR AUTOMATION. PROTECT WHAT MATTERS.
WITH ANSIBLE VAULT

“ Ansible Vault allows you to encrypt files and variables so sensitive information like passwords, API keys, and tokens stay secure while still enabling automation at scale. ”

- VERIQTA

WHY IT MATTERS

- ✓ Protect sensitive data in playbooks.
- ✓ Prevent credential leaks.
- ✓ Meet security and compliance standards.
- ✓ Enable safe collaboration.
- ✓ Support secret rotation and governance.

1 ANSIBLE VAULT OVERVIEW



- ✓ Built-in CLI tool to encrypt sensitive data.
- ✓ Encrypt entire files or individual variables.
- ✓ Works with playbooks, vars files, and secrets.
- ✓ AES-256 encryption (default).

COMMON USE CASES

- Database passwords
- API keys and tokens
- Cloud access keys
- TLS private keys
- Third-party service creds

SUPPORTED TARGETS

- Vars files (group_vars, host_vars)
- Playbooks
- Inventory files
- Custom data files
- Inline variables (with care)

2 VAULT PASSWORDS



- ✓ Vault needs a password (or multiple) to decrypt.
- ✓ Never hardcode passwords in scripts or repos.
- ✓ Use secure methods to store and provide passwords.
- ✓ Support multiple Vault IDs for different secrets.

USING VAULT PASSWORD FILE

```
# Create password file (secure permissions)
echo 'StrongVaultPass!2025' > ~/.vault_pass
chmod 600 ~/.vault_pass

# Use it
ansible-playbook site.yml --vault-password-file ~/.vault_pass
```

BEST PRACTICES

- Protect password files (600 perms).
- Store outside the repository.
- Use environment variables or secret managers.
- Rotate passwords regularly.

3 ENCRYPTED VARIABLES



- ✓ Encrypt entire vars files or individual values.
- ✓ Keep sensitive variables out of plain text.
- ✓ Decrypt automatically during playbook run.
- ✓ Example: group_vars/db.yml

ENCRYPT EXAMPLE

```
ansible-vault encrypt group_vars/db.yml
# File is now encrypted
$ANSIBLE_VAULT;1.1;AES256
663336363663... (encrypted content)
```

EDIT ENCRYPTED FILE

```
ansible-vault edit group_vars/db.yml
# Decrypts to temp file, opens editor,
re-encrypts on save
```

4 VAULT IDS



- ✓ Use multiple Vault IDs to separate domains.
- ✓ E.g., dev, prod, apps, teams, or environments.
- ✓ Choose the right ID for the right secret.
- ✓ Improves access control and auditability.

VAULT ID CONFIG

```
[defaults]
vault_identity_list = dev@~/.vault_dev, prod@~/.vault_prod

# Use specific ID
ansible-playbook site.yml --vault-id dev
ansible-playbook site.yml --vault-id prod
```

BEST PRACTICES

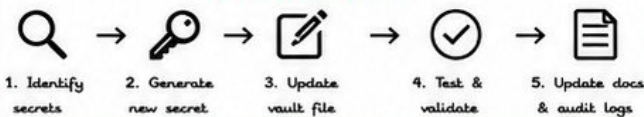
- One Vault ID per domain/team.
- Limit who has access to each password.
- Use clear naming (dev, prod, etc.).
- Document purpose of each ID.

5 SECRET ROTATION



- ✓ Rotate secrets regularly to reduce risk.
- ✓ Update encrypted files and dependent systems.
- ✓ Validate after rotation.
- ✓ Automate where possible.

ROTATION WORKFLOW



6 SECURITY BEST PRACTICES



- ✓ Use least privilege for access to Vault passwords.
- ✓ Never commit Vault passwords to version control.
- ✓ Encrypt before committing to Git.
- ✓ Use separate Vault IDs for separation of duties.
- ✓ Audit access and changes regularly.
- ✓ Enable 2FA on systems storing Vault passwords.



DO'S

- Store passwords securely.
- Encrypt all sensitive data.
- Review access regularly.
- Monitor for unauthorized access.



DON'TS

- Don't hardcode secrets.
- Don't share Vault passwords.
- Don't commit decrypted files.
- Don't ignore rotation.

7 USE VAULT SECURELY IN AUTOMATION



- ✓ Use environment variables in CI/CD.
- ✓ Use secret managers to inject Vault passwords.
- ✓ Avoid prompting for passwords in automation.
- ✓ Log carefully: never print secrets.

CI/CD EXAMPLE (GITLAB CI)

```
variables:
  ANSIBLE_VAULT_PASSWORD_FILE: /tmp/vault_pass

before_script:
  - echo "$VAULT_PASS" > /tmp/vault_pass
  - chmod 600 /tmp/vault_pass

script:
  - ansible-playbook site.yml
    --vault-password-file /tmp/vault_pass
```

8 QUICK VAULT COMMANDS



```
ansible-vault create file.yml # Create new encrypted file
ansible-vault edit file.yml # Edit encrypted file
ansible-vault encrypt file.yml # Encrypt existing file
ansible-vault decrypt file.yml # Decrypt file
ansible-vault view file.yml # View decrypted content
ansible-vault rekey file.yml # Change password
ansible-vault encrypt_string 'secret' # Encrypt inline string
```

SECRET MANAGEMENT CHECKLIST



- ✓ Identify all sensitive data
- ✓ Store passwords securely
- ✓ Restrict access
- ✓ Audit and monitor access
- ✓ Encrypt with Ansible Vault
- ✓ Use Vault IDs appropriately
- ✓ Rotate secrets regularly
- ✓ Document processes

★ PRO TIP: Treat secrets like production code—version, protect, rotate, and audit. Security is everyone's responsibility. - VERIQTA



ANSIBLE

VERIQTA



ERROR HANDLING

**BUILD RESILIENT PLAYBOOKS.
HANDLE FAILURE GRACEFULLY. AUTOMATE RECOVERY.**

“Great automation expects failure and handles it.
Error handling in Ansible helps you control failures,
apply logic, and recover automatically.”

- VERIQTA

WHY ERROR HANDLING?

- ✓ Prevent playbook crashes.
- ✓ Handle expected failures.
- ✓ Improve reliability.
- ✓ Enable safe automation at scale.
- ✓ Build self-healing workflows.

1 IGNORE_ERRORS



- ✓ Continue playbook execution even if the task fails.
- ✓ Use only for non-critical or best-effort tasks.
- ✓ Task will still be marked as failed, but play continues.

SYNTAX

```
- name: Try something risky
  command: /usr/bin/risky_command
  ignore_errors: yes
```

EXAMPLE

```
- name: Check optional service
  service:
    name: optional-service
    state: started
    ignore_errors: yes
```

2 FAILED_WHEN



- ✓ Define custom conditions that mark a task as failed.
- ✓ Useful when command returns 0 but result is not valid.
- ✓ Override Ansible's default failure behavior.

SYNTAX

```
- name: Validate disk space
  shell: df -h /
  register: disk
  failed_when: disk.stdout is search("[0-9]+%")
  and (disk.stdout | regex_search("[0-9]+%") | int
    < 80)
```

EXAMPLE

```
- name: Check API health
  uri:
    url: https://api/app/health
    return_content: yes
  register: result
  failed_when: result.json.status != "ok"
```

3 CHANGED_WHEN



- ✓ Define custom conditions that mark a task as changed.
- ✓ Helps keep reports clean and accurate.
- ✓ Prevents false "changed" status.

SYNTAX

```
- name: Run command
  command: /usr/bin/safe_cmd
  register: cmd
  changed_when: "updated" in cmd.stdout
```

EXAMPLE

```
- name: Check file content
  shell: cat /etc/app.conf
  register: content
  changed_when: false
```

4 BLOCK



- ✓ Group tasks together.
- ✓ Used with rescue and always for advanced error handling.
- ✓ If any task in block fails, control moves to rescue.

SYNTAX

```
- block:
  - task 1
  - task 2
  - task 3
```

EXAMPLE

```
- block:
  - name: Stop service
    service: name=app state=stopped
  - name: Update package
    apt: name=app state=latest
```

5 RESCUE



- ✓ Runs only when a task in the block fails.
- ✓ Use to recover, rollback, or notify.
- ✓ Receives the failed task context.

SYNTAX

```
- block:
  - task 1
  - task 2
  rescue:
    - rollback task
    - notify admin
```

EXAMPLE

```
- block:
  - name: Deploy app
    include_role: name=deploy_app
  rescue:
    - name: Rollback app
      include_role: name=rollback_app
    - name: Notify failure
      mail: to=ops@example.com
        subject="Deployment failed"
```

6 ALWAYS



- ✓ Runs always, regardless of success or failure.
- ✓ Use for cleanup, logging, notifications.
- ✓ Executes after block or rescue.

SYNTAX

```
- block:
  - task 1
  - task 2
  rescue:
    - recover
  always:
    - cleanup
    - notify
```

EXAMPLE

```
- block:
  - name: Perform operation
    command: /usr/bin/do_something
  rescue:
    - debug: msg="Failed!"
  always:
    - name: Cleanup temp files
      file: path=/tmp/app state=absent
```

7 RECOVERY WORKFLOWS



- ✓ Design self-healing playbooks.
- ✓ Validate, recover, retry, and notify.
- ✓ Keep systems reliable and consistent.

TYPICAL RECOVERY WORKFLOW



8 BEST PRACTICES



- ✓ Handle only expected failures.
- ✓ Don't ignore real issues.
- ✓ Log failures clearly.
- ✓ Keep recovery steps idempotent.
- ✓ Test failure scenarios regularly.
- ✓ Notify the right people.

DON'TS

- ✗ Don't ignore critical failures.
- ✗ Don't use ignore_errors blindly.
- ✗ Don't hide problems from monitoring.
- ✗ Don't skip notifications.
- ✗ Don't leave systems in unknown state.

ERROR HANDLING CHECKLIST



- ✓ Use block/rescue/always for risky operations.
- ✓ Define clear failure conditions (failed_when).
- ✓ Avoid ignore_errors for important tasks.
- ✓ Ensure recovery and rollback steps exist.

- ✓ Clean up resources in always.
- ✓ Log and notify on failures.
- ✓ Test failure scenarios.
- ✓ Keep playbooks idempotent.

★ **PRO TIP:** Expect failure. Handle gracefully. Recover automatically. That's how resilient automation is built!

- VERIQTA



ANSIBLE

VERIQTA TAGS

**RUN WHAT YOU NEED. SKIP WHAT YOU DON'T.
MAXIMUM CONTROL. CLEAN AUTOMATION.**

“Tags let you organize and control playbook execution.
Run specific parts, skip the rest.
Faster, safer, and made for production.”

- VERIQTA

WHY USE TAGS?

- ✓ Run only what is needed.
- ✓ Speed up deployments.
- ✓ Safer changes.
- ✓ Organize tasks logically.
- ✓ Improve debugging.
- ✓ Standardize automation.

1 TASK TAGGING



- ✓ Add tags to tasks, blocks, or roles.
- ✓ Tags can be one or many.
- ✓ Tags can be inherited in blocks and roles.

SYNTAX

```
- name: Install Nginx
  apt:
    name: nginx
    state: present
  tags:
    - packages
    - web
```

EXAMPLE (MULTIPLE TAGS)

```
- name: Start Nginx
  service:
    name: nginx
    state: started
  tags: [ web, services, prod ]
```

2 SELECTIVE EXECUTION



- ✓ Run only tasks with specific tags.
- ✓ Use --tags to include.
- ✓ Combine multiple tags with commas.
- ✓ Great for partial runs in production.

COMMANDS

```
# Run tasks with tag 'web'
ansible-playbook site.yml --tags web
# Run multiple tags
ansible-playbook site.yml --tags web,db
# Run tasks with either tag
ansible-playbook site.yml --tags web,db
# Use a variable
ansible-playbook site.yml --tags "{{ run_tags }}"
```

USE CASES

- Run only database migrations.
- Deploy only web services.
- Run monitoring tasks alone.
- Speed up testing.

3 SKIP TAGS



- ✓ Skip tasks with specific tags.
- ✓ Use --skip-tags to exclude.
- ✓ Useful for avoiding risky or unnecessary changes.

COMMANDS

```
# Skip tasks with tag 'debug'
ansible-playbook site.yml --skip-tags debug
# Skip multiple tags
ansible-playbook site.yml --skip-tags test,dev
# Skip tags using variable
ansible-playbook site.yml --skip-tags "{{ skip_tags }}"
```

USE CASES

- Skip debug tasks in production.
- Skip heavy tests.
- Avoid environment-specific tasks.

4 DEBUG TAGS



- ✓ Tag debug, troubleshoot, and verbose tasks.
- ✓ Run them only when needed.
- ✓ Keep playbooks clean in normal runs.

EXAMPLE

```
- name: Show variables
  debug:
    var: my_var
    tags: [ debug ]
- name: Print server facts
  debug:
    var: ansible_facts
    tags: [ debug, verbose ]
```

RUN WHEN NEEDED

```
# Run debug tasks only
ansible-playbook site.yml --tags debug

# Skip debug tasks
ansible-playbook site.yml \
  --skip-tags debug
```

5 ENVIRONMENT TAGS



- ✓ Tag tasks by environment.
- ✓ Run only what applies to dev, test, staging, or production.
- ✓ Keep playbooks reusable across environments.

EXAMPLE

```
- name: Configure app for Production
  template:
    src: app.conf.j2
    dest: /etc/app/app.conf
  tags: [ prod ]
- name: Configure app for Dev
  template:
    src: app-dev.conf.j2
    dest: /etc/app/app.conf
  tags: [ dev ]
```

COMMANDS

```
# Run only production tasks
ansible-playbook site.yml --tags prod
# Run only dev tasks
ansible-playbook site.yml --tags dev
# Skip prod tasks in non-prod
ansible-playbook site.yml --skip-tags prod
```

6 ENTERPRISE ORGANIZATION



- ✓ Define a tagging standard.
- ✓ Keep tags consistent across all playbooks.
- ✓ Document and enforce usage.

RECOMMENDED TAG CATEGORIES

```
• role:<role_name>      # e.g., role:web
• layer:<layer>         # e.g., layer:app
• env:<environment>     # e.g., env:prod
• type:<purpose>        # e.g., type:config
• critical              # e.g., critical
• compliance            # e.g., compliance
• debug                # troubleshooting
```

EXAMPLE TAGS

```
• role:web
• layer:application
• env:prod
• type:config
• type:deploy
• critical
• compliance
• debug
```

7 BEST PRACTICES



- ✓ Use meaningful, consistent tag names.
- ✓ Tag at task or block level.
- ✓ Avoid over-tagging small trivial tasks.
- ✓ Document tags in README.
- ✓ Review tags during code reviews.

DO'S

- ✓ Use clear naming.
- ✓ Keep tags standardized.
- ✓ Use categories.
- ✓ Combine tags when needed.
- ✓ Keep documentation updated.

DON'TS

- ✗ Don't use tags as a replacement for roles.
- ✗ Don't over-complicate naming.
- ✗ Don't leave unused tags.
- ✗ Don't forget to document.

8 QUICK REFERENCE

ACTION	COMMAND	EXAMPLE
Run specific tags	--tags	ansible-playbook site.yml --tags web
Run multiple tags	--tags	ansible-playbook site.yml --tags web,db
Skip specific tags	--skip-tags	ansible-playbook site.yml --skip-tags debug
Run all (default)	(no option)	ansible-playbook site.yml
List tasks with tags	--list-tags	ansible-playbook site.yml --list-tags
List tasks to run	--list-tasks	ansible-playbook site.yml --tags web --list-tasks

TAG-BASED EXECUTION WORKFLOW



1. Tag Tasks
Organize tasks with meaningful tags.



2. Select Tags
Choose tags to run or skip.



3. Execute
Run playbook with tags.



4. Validate
Verify results and health.



5. Iterate
Adjust tags and improve.

★ **PRO TIP:** Use tags to make your automation faster, safer, and more reliable. Run less. Achieve more.

- VERIQTA



YouTube
VERIQTA



GitHub
veriqta



LinkedIn
VERIQTA



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



ANSIBLE

DYNAMIC INVENTORY

DISCOVER. ORGANIZE. AUTOMATE.
INVENTORY THAT ADAPTS TO YOUR INFRASTRUCTURE.

“Dynamic Inventory allows Ansible to pull hosts and groups from external sources in real time. Your inventory is always accurate, up to date, and ready for automation.”

- VERIQTA ”

WHY DYNAMIC INVENTORY?

- ✓ No manual host updates.
- ✓ Auto-discover instances.
- ✓ Reflect real-time changes.
- ✓ Scale with cloud & infrastructure.
- ✓ Reduce errors and save time.
- ✓ Perfect for modern, ephemeral infrastructures.

1 WHAT IS DYNAMIC INVENTORY?



- ✓ Inventory is generated by scripts or plugins.
- ✓ Hosts are discovered from an external source.
- ✓ No static files to maintain.
- ✓ Supports auto-scaling and ephemeral environments.

BENEFITS

- Always up to date
- Handles large environments
- Works with cloud, containers, and virtualization
- Reduces human error
- Saves time and effort

HOW IT WORKS

1. Ansible calls inventory plugin
2. Plugin queries the source
3. Hosts & groups returned in JSON
4. Ansible uses data for execution

2 SUPPORTED DYNAMIC INVENTORY SOURCES



AWS
(EC2)



AZURE
(VM/ARM)



GCP
(Compute)



VMWARE
(vCenter)



KUBERNETES
(API)



OTHER
(OpenStack,
DigitalOcean,
NetBox, etc.)

INVENTORY PLUGIN TYPES

- Cloud Plugins - aws_ec2, azure_rm, google.cloud.gcp_compute
- Virtualization - vmware_vm_inventory
- Container - kubernetes.core.k8s
- Script / Executable - Any script that outputs JSON inventory
- Constructed - Combine multiple sources with rules

3 AWS DYNAMIC INVENTORY



- ✓ Uses aws_ec2 plugin
- ✓ Discovers EC2 instances
- ✓ Supports tags, filters, regions
- ✓ Auto groups by tags, VPC, security groups

```
aws_ec2.yml EXAMPLE
plugin: amazon.aws.aws_ec2
regions:
  - us-east-1
filters:
  instance-state-name: running
keyed_groups:
  - key: tags.Environment
    prefix: env_
  - key: placement.region
    prefix: region_
compose:
  ansible_host: public_ip_address
```

4 AZURE DYNAMIC INVENTORY



- ✓ Uses azure_rm plugin
- ✓ Discovers Azure VMs
- ✓ Supports resource groups, tags, subscriptions

```
azure_rm.yml EXAMPLE
plugin: azure.azcollection.azure_rm
auth_source: auto
include_vm_resource_groups:
  - production-rg
  - dev-rg
keyed_groups:
  - key: tags.Environment
    prefix: env_
  - key: location
    prefix: location_
```

5 GCP DYNAMIC INVENTORY



- ✓ Uses google.cloud.gcp_compute
- ✓ Discovers Compute Engine instances
- ✓ Supports projects, zones, labels

```
gcp_compute.yml EXAMPLE
plugin: google.cloud.gcp_compute
projects:
  - my-gcp-project
zones:
  - us-central1-a
filters:
  status: RUNNING
keyed_groups:
  - key: labels.environment
    prefix: env_
```

6 VMWARE DYNAMIC INVENTORY



- ✓ Uses vmware_vm_inventory plugin
- ✓ Connects to vCenter
- ✓ Discovers VMs and metadata
- ✓ Groups by datacenter, folder, tags

```
vmware.yml EXAMPLE
plugin: vmware.vmware_vm_inventory
hostname: vcenter.example.com
username: administrator@vsphere.local
password: |vault |
  $ANSIBLE_VAULT;1.1;AES256...
validate_certs: false
host_filters:
  - power_state: poweredOn
with_tags: true
```

7 KUBERNETES DYNAMIC INVENTORY



- ✓ Uses kubernetes.core.k8s plugin
- ✓ Discovers nodes, pods, services
- ✓ Works with labels & namespaces
- ✓ Great for containerized apps

```
k8s.yml EXAMPLE
plugin: kubernetes.core.k8s
host: https://kubernetes.default.svc
validate_certs: true
api_key: |vault |
  $ANSIBLE_VAULT;1.1;AES256...
namespaces:
  - default
  - production
  - monitoring
group_by:
  - key: labels['app']
```

8 INVENTORY PLUGINS OVERVIEW



- ✓ Many built-in inventory plugins
- ✓ Each plugin outputs JSON
- ✓ Ansible reads and builds inventory
- ✓ Place plugin config in inventory/ dir

COMMON PLUGINS

```
amazon.aws.aws_ec2
azure.azcollection.azure_rm
google.cloud.gcp_compute
community.vmware.vmware_vm_inventory
kubernetes.core.k8s
openstack.cloud.openstack
digitalocean.digital_ocean
netbox.netbox.nb_inventory
constructed (compose & merge)
```

9 ENTERPRISE ENVIRONMENTS - BEST PRACTICES

- ✓ Store inventory configs in version control.
- ✓ Use environment variables or Ansible Vault for secrets.
- ✓ Limit permissions (least privilege access).
- ✓ Use filters to limit discovered hosts.



SECURE CREDENTIALS
Use Vault / env vars.



USE FILTERS
Limit to what you need.



VERSION CONTROL
Track and review inventory changes.



MONITOR CHANGES
Keep inventory clean and controlled.



AUTOMATE & SCALE
Let inventory grow with your infra.

10 EXAMPLE: RUN PLAYBOOK WITH DYNAMIC INVENTORY



COMMAND

```
ansible-playbook -i inventory/aws_ec2.yml site.yml
```

EXAMPLE OUTPUT

```
PLAY [Web Servers] *****
TASK [Gathering Facts] *****
ok: [i-0abc123de456]
ok: [i-0def456abc789]
...
```



TIP

Use --list-hosts to see discovered hosts.
ansible-inventory -i inventory/aws_ec2.yml --list

★ PRO TIP: Dynamic Inventory keeps your automation accurate, scalable, and future-ready.

- VERIQTA



ANSIBLE

➤ VERIQTA ⚡



ANSIBLE FOR AWS

AUTOMATE. PROVISION. SECURE.
INFRASTRUCTURE AS CODE WITH ANSIBLE.

“ Ansible makes it simple to automate AWS infrastructure.
Use modules to build, configure, secure, and manage
resources at scale. ”

- VERIQTA

WHY ANSIBLE FOR AWS?

- ✓ No agent, simple and powerful.
- ✓ Infrastructure as Code.
- ✓ Idempotent and repeatable.
- ✓ Scalable and secure.
- ✓ Works across accounts and regions.
- ✓ Perfect for DevOps and Cloud Engineers.

1 AWS MODULES OVERVIEW



- ✓ Ansible provides powerful modules for AWS.
- ✓ Use AWS credentials or IAM roles.
- ✓ Supports all major AWS services.
- ✓ Idempotent - run multiple times safely.

COMMON REQUIREMENTS

- boto3 and botocore (AWS SDK for Python)
- AWS credentials (Access Key, Secret Key, Region)
- Permissions via IAM User or IAM Role
- ansible.builtin.aws modules collection

```
# Install AWS collection
ansible-galaxy collection install amazon.aws
```

2 KEY AWS SERVICES YOU CAN AUTOMATE WITH ANSIBLE



EC2
Instances



VPC
Networks



IAM
Users & Roles



S3
Storage



Security
Groups



ELB
Load Balancer

3 EC2 - MANAGE INSTANCES



- ✓ Launch, start, stop, terminate instances.
- ✓ Manage instance types, AMIs, key pairs.
- ✓ Tagging, monitoring, and more.

EXAMPLE: Launch EC2 Instance

```
- name: Launch EC2
  amazon.aws.ec2_instance:
    name: web-server
    instance_type: t3.micro
    image_id: ami-0c02fb55956c7d316
    key_name: my-key
    security_group: web-sg
    region: us-east-1
    wait: yes
```

4 VPC - MANAGE NETWORKS



- ✓ Create VPCs, subnets, route tables.
- ✓ Attach Internet Gateways.
- ✓ Manage NAT Gateways and Peering.

EXAMPLE: Create VPC

```
- name: Create VPC
  amazon.aws.ec2_vpc_net:
    name: my-vpc
    cidr_block: 10.10.0.0/16
    region: us-east-1
    tags:
      Name: my-vpc
```

5 IAM - USERS & ROLES



- ✓ Create users, groups, and roles.
- ✓ Attach managed or custom policies.
- ✓ Manage access keys and permissions.

EXAMPLE: Create IAM User

```
- name: Create IAM User
  amazon.aws.iam_user:
    name: devops-user
    groups: developers
    state: present
    create_access_key: yes
```

6 S3 - STORAGE AUTOMATION



- ✓ Create buckets, manage policies.
- ✓ Enable versioning and encryption.
- ✓ Upload or manage objects.

EXAMPLE: Create S3 Bucket

```
- name: Create S3 Bucket
  amazon.aws.s3_bucket:
    name: my-ansible-bucket
    region: us-east-1
    versioning: yes
    encryption: AES256
    state: present
```

7 SECURITY GROUPS



- ✓ Create and manage Security Groups.
- ✓ Allow/deny inbound and outbound rules.
- ✓ Attach to EC2 instances.

EXAMPLE: Create Security Group

```
- name: Create Security Group
  amazon.aws.ec2_security_group:
    name: web-sg
    description: Allow HTTP and SSH
    vpc_id: vpc-12345678
    rules:
      - proto: tcp
        ports: [22, 80]
      - cidr_ip: 0.0.0.0/0
```

8 ELB - LOAD BALANCING



- ✓ Create Application or Network Load Balancer.
- ✓ Configure listeners, target groups.
- ✓ Register instances and health checks.

EXAMPLE: Create ALB

```
- name: Create Application Load Balancer
  amazon.aws.elb_application_lb:
    name: my-alb
    subnets: [subnet-1234, subnet-5678]
    security_groups: [web-sg]
    scheme: internet-facing
    state: present
```

9 INFRASTRUCTURE AUTOMATION WORKFLOW



1. Define Inventory
Set AWS inventory
(static or dynamic).



3. Write Playbook
Use AWS and automate
your infrastructure.



4. Verify & Monitor
Validate resources
and monitor.



5. Repeat & Scale
Idempotent automation
at enterprise scale.

10 BEST PRACTICES

- ✓ Use IAM roles with least privilege.
- ✓ Store credentials securely (AWS CLI profile, Vault, or IAM Role).
- ✓ Use variables and templates for reusability.
- ✓ Tag all resources for ownership and cost tracking.
- ✓ Use consistent naming conventions.
- ✓ Enable logging and monitoring.
- ✓ Test playbooks in non-production first.
- ✓ Automate, document, and version control everything.



QUICK TIP

Ansible + AWS = Speed,
Consistency, Security,
and Scale.

☆ **PRO TIP:** Automate Everything. Save Time. Reduce Errors. Scale With Confidence.

- VERIQTA



YouTube
VERIQTA



GitHub
veriqta



LinkedIn
VERIQTA



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



ANSIBLE

ANSIBLE FOR KUBERNETES

AUTOMATE. DEPLOY. SCALE.
MANAGE KUBERNETES CLUSTERS WITH ANSIBLE.

“ Ansible provides powerful Kubernetes modules to manage applications, resources, and configurations declaratively. Infrastructure as code for modern container platforms. ”

- VERIQTA

WHY ANSIBLE FOR KUBERNETES?

- ✓ Agentless and simple.
- ✓ Use YAML, not custom languages.
- ✓ Idempotent and repeatable.
- ✓ Perfect for GitOps and CI/CD.
- ✓ Works across multiple clusters.
- ✓ Ideal for DevOps and SRE teams.

1 KUBERNETES MODULES OVERVIEW



- ✓ Use the `kubernetes.core` collection.
- ✓ Manage resources declaratively.
- ✓ Supports create, update, patch, delete.
- ✓ Idempotent - safe to run multiple times.
- ✓ Works with any Kubernetes distribution.

INSTALL COLLECTION

`ansible-galaxy collection install kubernetes.core`

2 CORE KUBERNETES MODULES



k8s
Resource
Module
(generic)



k8s_info
Get resource
information



k8s_scale
Scale
deployments



k8s
Delete
resources



k8s_facts
Get cluster
facts



k8s
Validate
manifests

CONNECTION REQUIREMENT

- ✓ Kubernetes config file (`~/.kube/config`) or Service account token with correct permissions.
- Set environment variable:
`export K8S_AUTH_KUBECONFIG=~/.kube/config`

3 DEPLOYMENTS



- ✓ Manage Deployments and ReplicaSets.
- ✓ Update images and replicas.
- ✓ Rolling updates and rollbacks.

EXAMPLE

```
- name: Create Deployment
  kubernetes.core.k8s:
    state: present
    definition:
      apiVersion: apps/v1
      kind: Deployment
      metadata:
        name: webapp
        namespace: default
      spec:
        replicas: 3
        selector:
          matchLabels:
            app: webapp
        template:
          metadata:
            labels:
              app: webapp
          spec:
            containers:
              - name: webapp
                image: nginx:latest
                ports:
                  - containerPort: 80
```

4 SERVICES



- ✓ Create and manage Services.
- ✓ Support ClusterIP, NodePort, LoadBalancer.
- ✓ Expose applications to users.

EXAMPLE

```
- name: Create Service
  kubernetes.core.k8s:
    state: present
    definition:
      apiVersion: v1
      kind: Service
      metadata:
        name: webapp-service
        namespace: default
      spec:
        type: LoadBalancer
        selector:
          app: webapp
        ports:
          - port: 80
            targetPort: 80
```

5 CONFIGMAPS



- ✓ Store non-sensitive configuration data.
- ✓ Inject into pods as files or env vars.
- ✓ Update without rebuilding images.

EXAMPLE

```
- name: Create ConfigMap
  kubernetes.core.k8s:
    state: present
    definition:
      apiVersion: v1
      kind: ConfigMap
      metadata:
        name: app-config
        namespace: default
      data:
        APP_MODE: "production"
        LOG_LEVEL: "info"
```

6 SECRETS



- ✓ Store sensitive data securely.
- ✓ Support opaque and custom types.
- ✓ Mount as files or environment variables.

EXAMPLE

```
- name: Create Secret
  kubernetes.core.k8s:
    state: present
    definition:
      apiVersion: v1
      kind: Secret
      metadata:
        name: db-credentials
        namespace: default
      type: Opaque
      data:
        username: YWRtaXQ= # admin (base64)
        password: cGFzc3dvcmQ= # password (base64)
```

7 NAMESPACES & CONTEXT



- ✓ Manage namespaces.
- ✓ Switch contexts between clusters.
- ✓ Organize resources by environment.

EXAMPLE

```
- name: Create namespace
  kubernetes.core.k8s:
    state: present
    definition:
      apiVersion: v1
      kind: Namespace
      metadata:
        name: staging
```

TIP: Use `--limit` or `inventory` with multiple clusters and contexts for multi-cluster ops.

8 HELM INTEGRATION



- ✓ Install, upgrade and uninstall Helm charts.
- ✓ Manage releases declaratively.
- ✓ Perfect for complex applications.

EXAMPLE

```
- name: Install Nginx Ingress via Helm
  kubernetes.core.helm:
    name: nginx-ingress
    chart_ref: ingress-nginx/ingress-nginx
    release_namespace: ingress-nginx
    create_namespace: yes
    values:
      controller:
        service:
          type: LoadBalancer
```

Requires: `helm` binary on control node and `kubernetes.core` collection.

9 KUBERNETES AUTOMATION WORKFLOW



1. Define
Resources
(YAML)



2. Write
Ansible Playbook



3. Apply to
Cluster



4. Verify &
Monitor



5. Iterate &
Improve

10 BEST PRACTICES

- ✓ Use least-privilege Service Accounts.
- ✓ Store `kubeconfig`/credentials securely (Vault).
- ✓ Use version control for all manifests and playbooks.
- ✓ Test playbooks in non-production environments.
- ✓ Enable logging and monitor changes.
- ✓ Keep playbooks idempotent and modular.



QUICK TIP

Ansible + Kubernetes
= Consistent, Secure
and Scalable
Infrastructure
Automation.

★ PRO TIP: Automate Everything. Save Time. Reduce Errors. Scale With Confidence.

- VERIQTA



ANSIBLE



ANSIBLE FOR DOCKER

AUTOMATE. CONTAINERIZE. SCALE.
MANAGE DOCKER RESOURCES WITH ANSIBLE.

“ Ansible's Docker modules let you manage containers, images, networks, volumes, and Docker Compose applications idempotently and at scale.

- VERIQTA ”

WHY ANSIBLE FOR DOCKER?

- ✓ Agentless and lightweight.
- ✓ Idempotent and repeatable.
- ✓ Consistent across environments.
- ✓ Works with Docker and Docker Compose.
- ✓ Ideal for DevOps and Platform Engineering.

1 DOCKER MODULES OVERVIEW



- ✓ Manage Docker lifecycle with Ansible.
- ✓ Supports Docker Engine API.
- ✓ Idempotent - safe to run multiple times.
- ✓ Works on Linux and compatible systems.

INSTALL COLLECTION

ansible-galaxy collection install community.docker

2 KEY DOCKER MODULES



docker_image
Manage
Images



docker_container
Manage
Containers



docker_network
Manage
Networks



docker_volume
Manage
Volumes



docker_compose
Run Docker
Compose apps



docker_prune
Cleanup
resources

CONNECTION REQUIREMENT

- ✓ Docker must be installed and running.
- ✓ ansible user must have permission to access Docker socket.
- ✓ Default: unix:///var/run/docker.sock
- ✓ Can configure with environment variable or module parameter.

3 DOCKER INSTALLATION



- ✓ Install Docker Engine.
- ✓ Ensure service is running.
- ✓ Enable and start at boot.

EXAMPLE: Install Docker

```
- name: Install Docker
  package:
    name: docker.io
    state: present
- name: Start Docker Service
  service:
    name: docker
    state: started
    enabled: yes
- name: Add user to docker group
  user:
    name: "{{ ansible_user }}"
    groups: docker
    append: yes
```

4 IMAGES



- ✓ Pull images from registry.
- ✓ Build images from Dockerfile.
- ✓ Remove unused images.

EXAMPLE: Pull Image

```
- name: Pull Nginx Image
  community.docker.docker_image:
    name: nginx
    tag: latest
    source: pull
- name: Build Custom Image
  community.docker.docker_image:
    name: myapp
    build:
      path: /opt/myapp
      tag: v1.0.0
    source: build
```

5 CONTAINERS



- ✓ Run, stop, start, restart containers.
- ✓ Manage ports, env, volumes.
- ✓ Ensure desired state.

EXAMPLE: Run Container

```
- name: Run Nginx Container
  community.docker.docker_container:
    name: web01
    image: nginx:latest
    state: started
    restart_policy: unless-stopped
    ports:
      - "80:80"
    volumes:
      - /opt/web:/usr/share/nginx/html
```

6 NETWORKS



- ✓ Create custom networks.
- ✓ Connect containers.
- ✓ Isolate and secure traffic.

EXAMPLE: Create Network

```
- name: Create App Network
  community.docker.docker_network:
    name: app-net
    driver: bridge
    state: present
```

EXAMPLE: Connect Container

```
- name: Connect Container
  community.docker.docker_container:
    name: web01
    networks:
      - name: app-net
```

7 VOLUMES



- ✓ Create and manage volumes.
- ✓ Persist container data.
- ✓ Support bind mounts too.

EXAMPLE: Create Volume

```
- name: Create Data Volume
  community.docker.docker_volume:
    name: app-data
    state: present
```

EXAMPLE: Use Volume in Container

```
- name: Run Container with Volume
  community.docker.docker_container:
    name: db01
    image: mysql:8
    state: started
    volumes:
      - app-data:/var/lib/mysql
    env:
      MYSQL_ROOT_PASSWORD: secret
```

8 DOCKER COMPOSE



- ✓ Deploy multi-container applications.
- ✓ Use docker-compose.yml.
- ✓ Manage lifecycle with ease.

EXAMPLE: Run Compose Application

```
- name: Deploy Application with Compose
  community.docker.docker_compose:
    project_src: /opt/myapp
    files:
      - docker-compose.yml
    state: present
    pull: yes
    restarted: yes
    remove_orphans: yes
```

9 COMMON DOCKER AUTOMATION PATTERNS



- ✓ Standardize container deployments.
- ✓ Blue/Green and Rolling updates.
- ✓ Cleanup unused resources.

PATTERN	DESCRIPTION
Idempotent Deployments	Ensure desired container state every run.
Rolling Updates	Update containers with zero downtime.
Health Checks	Verify container health and auto-recover.
Resource Cleanup	Remove stopped containers, unused images, and networks.
Multi-Env Deployments	Use inventory groups for dev, staging, prod.

10 INFRASTRUCTURE AUTOMATION WORKFLOW



1. Define
Requirements



2. Write
Playbook



3. Pull Images &
Create Resources



4. Run &
Configure



5. Verify &
Monitor

11 BEST PRACTICES

- ✓ Use specific image tags (avoid latest).
- ✓ Use restart policies for resiliency.
- ✓ Store playbooks in version control.
- ✓ Use variables and templates.
- ✓ Log and monitor Docker activities.



QUICK TIP

Ansible + Docker =
Consistent, Portable,
and Scalable
Applications.

★ PRO TIP: Automate Everything. Save Time. Reduce Errors. Scale With Confidence.

- VERIQTA



YouTube
VERIQTA



GitHub
veriqta



LinkedIn
VERIQTA



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



ANSIBLE

VERIQTA

NETWORK AUTOMATION

AUTOMATE. STANDARDIZE. VALIDATE.
BUILD RELIABLE, COMPLIANT, AND SCALABLE NETWORKS.

“ Ansible makes network automation simple and consistent.
Connect to devices, back up configurations, enforce standards,
validate changes, and keep your enterprise network reliable.
- VERIQTA ”



WHY NETWORK AUTOMATION?

- ✓ Eliminate manual configuration errors.
- ✓ Ensure compliance and consistency.
- ✓ Back up and version configurations.
- ✓ Validate changes before and after.
- ✓ Scale network operations efficiently.
- ✓ Ideal for Enterprise Networking teams.

1 NETWORK AUTOMATION OVERVIEW



- ✓ Automate configuration, monitoring, and compliance across network devices.
- ✓ Support multi-vendor environments.
- ✓ Reduce human error and increase speed.
- ✓ Enforce standards and improve visibility.
- ✓ Integrate with CI/CD and ITSM workflows.

COMMON USE CASES

- Configuration backup and restore
- Mass configuration changes
- Compliance and security enforcement
- Network validation and health checks
- Zero-touch provisioning
- Reporting and documentation

2 SUPPORTED NETWORK VENDORS



- CISCO**
- ✓ IOS / IOS-XE
 - ✓ NX-OS
 - ✓ ASA / Firepower
 - ✓ WLC



- JUNIPER**
- ✓ Junos OS
 - ✓ SRX Series
 - ✓ EX / QFX Series



- ARISTA**
- ✓ EOS
 - ✓ CloudVision
 - ✓ Integration



- OTHERS**
- ✓ FS, Palo Alto
 - ✓ Fortinet
 - ✓ Check Point
 - ✓ And more...

CONNECTION METHODS



SSH
(Preferred)



NETCONF
(Structured)



REST / API
(Modern)



LOCAL
(On-box)

3 CISCO AUTOMATION



- ✓ Use cisco.ios / cisco.iosxr modules.
- ✓ Backup, configure, and validate.
- ✓ Supports facts gathering and diffs.

EXAMPLE: BACKUP RUNNING CONFIG

```
- name: Backup Cisco IOS running config
  cisco.ios.ios_config:
    backup: yes
    backup_options:
      filename: "{{ inventory_hostname }}-{{
        ansible_date_time.date }}.cfg"
      dir_path: ./backup
    backup: running_config
```

EXAMPLE: PUSH CONFIGURATION

```
- name: Push configuration to Cisco device
  cisco.ios.ios_config:
    lines:
      - interface GigabitEthernet1/0/1
      - description Uplink to Core
      - ip address 10.0.1.1 255.255.255.0
      - no shutdown
    save_when: changed
```

4 JUNIPER AUTOMATION



- ✓ Use junipernetworks.junos modules.
- ✓ Backup, configure, and manage states.
- ✓ NETCONF support for structured mgmt.

EXAMPLE: BACKUP CONFIGURATION

```
- name: Backup Junos configuration
  junipernetworks.junos.junos_config:
    backup: yes
    backup_options:
      filename: "{{ inventory_hostname }}-{{
        ansible_date_time.date }}.cfg"
      dir_path: ./backup
    backup: set
```

EXAMPLE: LOAD CONFIGURATION

```
- name: Load configuration on Junos
  junipernetworks.junos.junos_config:
    src: configs/junos-set.conf
    load: set
    commit: yes
    confirm: 5
```

5 ARISTA AUTOMATION



- ✓ Use arista.eos modules.
- ✓ Automate EOS configuration.
- ✓ Integrate with CloudVision.

EXAMPLE: BACKUP CONFIGURATION

```
- name: Backup Arista EOS configuration
  arista.eos.eos_config:
    backup: yes
    backup_options:
      filename: "{{ inventory_hostname }}-{{
        ansible_date_time.date }}.cfg"
      dir_path: ./backup
    backup: running_config
```

EXAMPLE: CONFIGURE INTERFACE

```
- name: Configure interface on Arista
  arista.eos.eos_config:
    lines:
      - interface Ethernet11
      - description Server Link
      - switchport mode trunk
      - no shutdown
    save_when: changed
```

6 CONFIGURATION BACKUP



- ✓ Automate regular backups.
- ✓ Store in Git, S3, or centralized repo.
- ✓ Version control and rollback ready.

EXAMPLE: SCHEDULED BACKUP PLAY

```
- hosts: network
  gather_facts: no
  tasks:
    - name: Backup device configuration
      cisco.ios.ios_config:
        backup: yes
        backup_options:
          filename: "{{ inventory_hostname }}-{{
            ansible_date_time.date }}.cfg"
          dir_path: ./backup
        backup: running_config
```

7 COMPLIANCE ENFORCEMENT



- ✓ Enforce security standards.
- ✓ Compare running vs desired state.
- ✓ Automated remediation.

EXAMPLE: ENFORCE NTP SERVER

```
- name: Ensure NTP server is configured
  cisco.ios.ios_config:
    lines:
      - ntp server 10.10.10.10
    parents: ntp
    match: line
    replace: line
    register: result
  - debug:
      msg: "Configuration updated"
    when: result.changed
```

8 NETWORK VALIDATION



- ✓ Validate before and after changes.
- ✓ Check interfaces, routes, and neighbors.
- ✓ Ensure network health and reachability.

EXAMPLE: VALIDATE INTERFACE STATUS

```
- name: Get interface facts
  cisco.ios.ios_facts:
    gather_subset:
      - interfaces
    register: intf

- name: Show interface summary
  debug:
    msg: "{{ intf.ansible_facts.ansible_net_interfaces }}"
```

9 ENTERPRISE NETWORKING – BEST PRACTICES



USE VERSION CONTROL
Store configurations
in Git for history
and rollback.



SECURE ACCESS
Use SSH keys, AAA,
role-based access,
and vault for secrets.



STANDARDIZE
Use templates,
roles, and naming
conventions.



VALIDATE CHANGES
Always validate
before and after
changes.



MONITOR & ALERT
Integrate with
monitoring and
alerting tools.



AUTOMATE CONTINUOUSLY
Integrate with CI/CD
and automate
network lifecycle.

10 SAMPLE WORKFLOW



1. Inventory
Load devices
(Cisco, Juniper, Arista)



2. Connect
Establish secure
connection



3. Backup
Backup running
configurations



4. Enforce
Push standard
configurations



5. Validate
Validate changes
and compliance



6. Report
Generate reports
and store logs



QUICK TIP
Automate small,
validate always,
backup everything,
and document
always.

PRO TIP: Automate network operations. Ensure compliance. Deliver reliable and secure enterprise networks.

- VERIQTA



ANSIBLE

VERIQTA

CI/CD INTEGRATION

AUTOMATE. INTEGRATE. DEPLOY.
SEAMLESS CI/CD PIPELINES WITH ANSIBLE.



WHY CI/CD INTEGRATION?

- ✓ Faster and consistent deployments
- ✓ Reduce manual errors
- ✓ Enable infrastructure as code
- ✓ Improve collaboration
- ✓ Rollback quickly and safely
- ✓ Deliver value continuously

“ Ansible integrates with leading CI/CD platforms to deliver consistent, automated, and reliable deployments at scale. Code to production with confidence. ”

- VERIQTA

1 OVERVIEW



- ✓ CI: Build, test, and validate code.
- ✓ CD: Deploy and operate code.
- ✓ Ansible automates provisioning, configuration, and deployments.
- ✓ Works across any environment.

INTEGRATION BENEFITS

- Automate infrastructure and application deployments.
- Enforce consistency across environments.
- Enable repeatable and version-controlled deployments.
- Support blue/green, rolling, and canary deployments.
- Integrate testing, security, and compliance checks.

2 CI/CD PLATFORMS SUPPORTED



GitHub Actions

- ✓ YAML based
- ✓ Native GitHub integrations
- ✓ Marketplace integrations



GitLab CI

- ✓ .gitlab-ci.yml
- ✓ Built-in runners
- ✓ Strong DevSecOps features



Jenkins

- ✓ Highly extensible
- ✓ Pipeline as Code
- ✓ Large plugin ecosystem



Azure DevOps

- ✓ YAML pipelines
- ✓ Microsoft ecosystem
- ✓ Advanced release management

HOW ANSIBLE FITS IN



Code Commit



Build & Test



Ansible Playbook



Deploy



Monitor

3 GITHUB ACTIONS + ANSIBLE



- ✓ Use Ansible in workflows.
- ✓ Store playbooks in repo.
- ✓ Use self-hosted or GitHub-hosted runners.
- ✓ Secure with secrets and environments.

EXAMPLE WORKFLOW (.github/workflows/deploy.yml)

```
- name: Deploy with Ansible
  hosts: servers
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Install Ansible
      run: sudo pip install ansible
    - name: Run Ansible Playbook
      run: ansible-playbook -i inventory.ini site.yml
  env:
    ANSIBLE_HOST_KEY_CHECKING: False
```

4 GITLAB CI + ANSIBLE



- ✓ Use .gitlab-ci.yml to define pipeline.
- ✓ Leverage GitLab runners.
- ✓ Manage environments and approvals.
- ✓ Integrate variables and vaults.

EXAMPLE (.gitlab-ci.yml)

```
deploy:
  stage: deploy
  image: python:3.11
  script:
    - pip install ansible
    - ansible-playbook -i inventory.ini site.yml
  environment:
    name: production
    url: https://app.example.com
  only:
    - main
```

5 JENKINS + ANSIBLE



- ✓ Use Pipeline or Freestyle jobs.
- ✓ Ansible via shell, Docker, or plugins.
- ✓ Manage credentials securely.
- ✓ Approvals and multi-stage releases.

EXAMPLE (JENKINS PIPELINE)

```
pipeline {
  agent any
  stages {
    stage('Deploy') {
      steps {
        sh 'ansible-playbook -i inventory.ini site.yml'
      }
    }
  }
}
```

6 AZURE DEVOPS + ANSIBLE



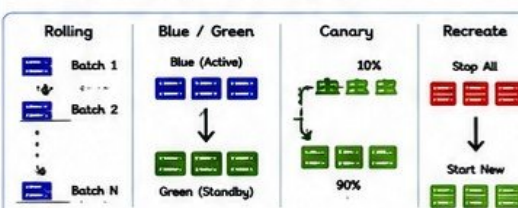
- ✓ Use YAML pipelines.
- ✓ Ansible via Azure CLI or scripts.
- ✓ Variable groups and service connections.
- ✓ Approvals and gated deployments.

EXAMPLE (azure-pipelines.yml)

```
- stage: Deploy
  jobs:
    - job: AnsibleDeploy
      steps:
        - task: UsePythonVersion@0
          inputs:
            versionSpec: '3.x'
        - script: |
            pip install ansible
            ansible-playbook -i inventory.ini site.yml
            displayName: 'Run Ansible Playbook'
```

7 AUTOMATED DEPLOYMENT STRATEGIES

- ✓ Rolling Deployment: Update servers in batches.
- ✓ Blue/Green Deployment: Zero downtime.
- ✓ Canary Deployment: Test with a small subset.
- ✓ Recreate Deployment: Replace all at once.



8 PIPELINE INTEGRATION PATTERNS



- ✓ Infrastructure Provisioning
- ✓ Application Deployment
- ✓ Configuration Management
- ✓ Security & Compliance Checks
- ✓ Database Migrations
- ✓ Post-Deployment Validation

PATTERN FLOW

Code → Build → Test →
Security Scan → Ansible Deploy →
Validate → Notify → Monitor

9 BEST PRACTICES



Store Ansible code
in version control.



Use Ansible Vault
for sensitive data.



Keep playbooks
idempotent.



Test playbooks
in CI before deploy.



Use environment
specific inventories.



Enable logging
and notifications.



Implement approvals
for production.



QUICK TIP

Automate everything
you can, validate
everything you deploy,
and monitor
everything always.

10 END-TO-END CI/CD PIPELINE EXAMPLE



1. Code Commit
Push changes



2. Build & Test
Run tests &
quality checks



3. Security Scan
Scan code &
dependencies



4. Ansible Deploy
Run playbooks
Deploy changes



5. Validate
Health checks
Smoke tests



6. Notify
Alert teams
& update status



7. Monitor
Track metrics
& logs closely

★ **PRO TIP:** Integrate early. Automate consistently. Deliver continuously. Improve relentlessly.

- VERIQTA



YouTube
VERIQTA



GitHub
veriqta



LinkedIn
VERIQTA



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



ANSIBLE



ANSIBLE PERFORMANCE

OPTIMIZE. ACCELERATE. SCALE.

BUILD HIGH-PERFORMING AUTOMATION AT ENTERPRISE SCALE.

“ High performance automation is not about speed alone. It is about running the right tasks, on the right systems, with the right resources, at the right time.

– VERIQTA ”

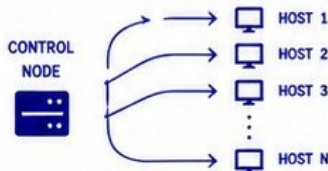
WHY PERFORMANCE MATTERS?

- ✓ Faster execution across thousands of systems.
- ✓ Lower control node load and cost.
- ✓ Consistent execution at scale.
- ✓ Improved reliability and timeouts.
- ✓ Better user experience.
- ✓ Enables true enterprise automation.

1 PARALLELISM OVERVIEW



- ✓ Ansible runs tasks in parallel.
- ✓ More hosts = more speed.
- ✓ Controlled by forks and strategy.
- ✓ Balance speed with stability.



TIP: More forks increase speed but also increase CPU, memory, and network usage.

2 FORKS (CONCURRENCY)



- ✓ Forks = number of parallel worker processes.
- ✓ Default: 5
- ✓ Set globally in ansible.cfg or via CLI.
- ✓ Higher forks = higher parallelism.

CONFIGURATION

[defaults]
forks = 50
Example: 50 parallel processes

BEST PRACTICES

- Start with forks = 20-50
- Increase gradually.
- Monitor control node resources.

3 STRATEGY PLUGINS



- ✓ Control how tasks are executed.
- ✓ Default: linear
- ✓ Other strategies improve performance for specific use cases.

STRATEGY	DESCRIPTION
linear	Default. Host-by-host in batches.
free	Host runs ahead without waiting for others.
host_pinned	Keeps tasks on same host order.
debug	Step-by-step execution.

EXAMPLE

ansible-playbook site.yml --strategy free

4 FACT CACHING



- ✓ Avoids gathering facts every run.
- ✓ Improves performance significantly.
- ✓ Multiple caching backends available.
- ✓ Enable fact caching globally.

CONFIGURATION (redis example)

[defaults]
fact_caching = redis
fact_caching_connection = 127.0.0.1:6379:0
fact_caching_timeout = 86400 # 24 hours

SUPPORTED BACKENDS

jsonfile, redis, memcached, yaml, database

5 SSH OPTIMIZATION



- ✓ SSH is the transport engine.
- ✓ Optimize connections for speed.
- ✓ Reuse connections and enable pipelining.

CONFIGURATION

[ssh_connection]
pipelining = True
ssh_control_path = ~/.ansible/cp/%%h-%%r
ssh_args = -o ControlMaster=auto -o ControlPersist=60s

BEST PRACTICES

- Enable pipelining.
- Use SSH multiplexing (ControlPersist).
- Use key-based authentication.

6 LARGE INVENTORIES



- ✓ Large inventories slow down execution.
- ✓ Use dynamic inventories.
- ✓ Limit target hosts.
- ✓ Avoid unnecessary facts.

BEST PRACTICES

- Use patterns: web*, db*, tag_Env_Prod
- Use --limit to reduce scope.
- Use fact caching.
- Disable facts for non-essential tasks.
- Use dynamic inventories (cloud plugins).

7 SCALING AUTOMATION



- ✓ Scale by architecture and design.
- ✓ Distribute load across control nodes.
- ✓ Use AWX/Automation Controller.
- ✓ Modularize playbooks and roles.

SCALING PATTERNS

Multiple Control Nodes (Active/Active)
AWX / Automation Controller
Execution Environments
Dynamic Inventory + Auto Discover
Event Driven Automation

8 PERFORMANCE TUNING CHECKLIST



- ✓ Increase forks appropriately.
- ✓ Enable pipelining and SSH multiplexing.
- ✓ Use fact caching.
- ✓ Use strategy plugins when needed.
- ✓ Limit target hosts.
- ✓ Optimize playbooks and roles.
- ✓ Monitor control node resources.
- ✓ Use async and poll for long tasks.

9 MONITORING & METRICS



- ✓ Measure and improve continuously.
- ✓ Use Ansible callback plugins.
- ✓ Monitor control node performance.
- ✓ Track execution time per task.

USEFUL TOOLS

- profile_tasks callback plugin
- timer callback plugin
- AWX/Controller Analytics
- System monitoring (CPU, RAM, IO, Network)

10 PERFORMANCE OPTIMIZATION WORKFLOW



1. Define
Scope & Goals



2. Optimize
Inventory



3. Tune
Configuration



4. Execute
& Measure



5. Analyze
Results



6. Iterate
& Improve



QUICK TIP

Start small.
Measure everything.
Optimize continuously.
Scale with confidence.

★ PRO TIP: Performance is a feature. Build it in early. Optimize always. Scale fearlessly.

– VERIQTA



ANSIBLE

VERIQTA



SECURITY HARDENING

AUTOMATE. ENFORCE. PROTECT.
BUILD SECURE, COMPLIANT, AND RESILIENT SYSTEMS.

“ Security is not a one-time task.

It is a continuous process of hardening, monitoring,
and improving your environment.

– VERIQTA ”

WHY SECURITY HARDENING?

- ✓ Reduce attack surface.
- ✓ Enforce security baselines.
- ✓ Meet compliance requirements.
- ✓ Automate repeatable controls.
- ✓ Improve visibility and auditability.
- ✓ Protect data and systems.
- ✓ Build enterprise-grade security.

1 CIS COMPLIANCE



- ✓ Follow CIS Benchmarks.
- ✓ Automate compliance checks.
- ✓ Generate compliance reports.
- ✓ Enforce security baselines.

EXAMPLE: CIS BENCHMARK (RHEL 9)

```
- name: Run CIS Benchmark
  ansible.builtin.include_role:
    name: ansible-role-cis
  vars:
    cis_level: 1
    cis_profile: "rhel9"
    generate_report: true
    report_path: /var/reports/cis/
```

COMMON CIS DOMAINS

- Account Policies
- Services
- Network Configuration
- Logging and Auditing
- Access, Authentication, Authorization
- System Maintenance

2 LINUX HARDENING



- ✓ Harden OS configuration.
- ✓ Disable unnecessary services.
- ✓ Secure kernel and sysctl.
- ✓ Enforce secure file permissions.

EXAMPLE: DISABLE UNUSED SERVICES

```
- name: Disable unused services
  ansible.builtin.systemd:
    name: "{{ item }}"
    enabled: false
    state: stopped
  loop:
    - telnet.socket
    - rsh.socket
    - rpcbind.service
    - avahi-daemon.service
```

HARDENING AREAS

- Kernel Parameters (sysctl)
- Service Management
- File Permissions
- Login Banners
- Audit & Logging
- Time Synchronization

3 SSH SECURITY



- ✓ Disable root login.
- ✓ Use key-based authentication.
- ✓ Enforce strong ciphers.
- ✓ Limit user access.

EXAMPLE: SECURE SSH CONFIG

```
- name: Configure SSH securely
  ansible.builtin.lineinfile:
    path: /etc/ssh/sshd_config
    regexp: "{{ item.key }}"
    line: "{{ item.key }} {{ item.value }}"
  loop:
    - { key: "PermitRootLogin", value: "no" }
    - { key: "PasswordAuthentication", value: "no" }
    - { key: "ChallengeResponseAuthentication", value: "no" }
    - { key: "MaxAuthTries", value: "3" }
    - { key: "AllowUsers", value: "adminops" }
```

RECOMMENDED SETTINGS

- PermitRootLogin no
- PasswordAuthentication no
- MaxAuthTries 3
- AllowUsers <specific_users>
- Use strong algorithms (ed25519, ecdsa)

4 USER MANAGEMENT



- ✓ Create and manage users.
- ✓ Enforce strong password policies.
- ✓ Use SSH keys and sudo policies.
- ✓ Disable inactive accounts.

EXAMPLE: CREATE SECURE USER

```
- name: Create user with SSH key
  ansible.builtin.user:
    name: devops
    shell: /bin/bash
    groups: wheel
    append: true
    create_home: true
    state: present

- name: Install SSH key
  ansible.posix.authorized_key:
    user: devops
    key: "{{ lookup('file', 'id_rsa.pub') }}"
```

BEST PRACTICES

- Use least privilege (sudo)
- Strong password or key auth
- Disable inactive/old accounts
- Enforce password aging

5 FIREWALL AUTOMATION



- ✓ Automate firewall rules.
- ✓ Allow only required ports.
- ✓ Use stateful rules.
- ✓ Ensure logging and drop defaults.

EXAMPLE: CONFIGURE FIREWALL (FIREWALLD)

```
- name: Allow required services
  ansible.posix.firewalld:
    service: "{{ item }}"
    permanent: true
    state: enabled
  loop:
    - ssh
    - https
    - http
  notify: reload firewall

- name: Set default policy
  ansible.posix:
    zone: public
    target: DROP
```

FIREWALL BEST PRACTICES

- Default deny (DROP)
- Allow only required services
- Log dropped packets
- Regular rule review

6 PATCH MANAGEMENT



- ✓ Automate OS patching.
- ✓ Schedule maintenance windows.
- ✓ Test before production rollout.
- ✓ Track patch compliance.

EXAMPLE: PATCH RHEL SYSTEMS

```
- name: Update all packages
  ansible.builtin.dnf:
    name: "*"
    state: latest
    register: update_result
    notify: reboot if needed

- name: Reboot if kernel updated
  ansible.builtin.reboot:
    reboot_timeout: 600
    when: update_result is changed
```

BEST PRACTICES

- Use approved repositories
- Test patches in staging
- Automate and schedule updates
- Reboot when required
- Maintain rollback plan

7 ENTERPRISE SECURITY AUTOMATION



- ✓ Enforce policies at scale.
- ✓ Continuous compliance.
- ✓ Centralized reporting.
- ✓ Integrate with security tools.

AUTOMATION EXAMPLES

- CIS Benchmark Enforcement
- User & Access Management
- Patch Management
- Firewall & Network Controls
- SSH Hardening
- File Integrity Monitoring
- Audit Log Collection
- Vulnerability Scanning Integration

BENEFITS

- Consistent security posture
- Reduced manual effort
- Faster incident response
- Audit-ready reports
- Stronger compliance posture

8 ENTERPRISE SECURITY PRINCIPLES



- ✓ Defense in depth.
- ✓ Least privilege access.
- ✓ Secure by default.
- ✓ Monitor and improve.

CORE PRINCIPLES

- Identify
- Protect
- Detect
- Respond
- Recover

COMPLIANCE FRAMEWORKS

- CIS Benchmarks
- NIST SP 800-53
- ISO 27001
- PCI DSS
- SOC 2

9 SECURITY HARDENING WORKFLOW



1. Assess
Evaluate current
security posture

2. Plan
Define hardening
standards

3. Automate
Use Ansible to enforce
security controls

4. Validate
Verify compliance
and test

5. Report
Generate compliance
reports

6. Monitor
Continuously monitor
and improve



QUICK TIP

Automate security
hardening early and
continuously. Security
is a journey, not
a destination.

☆ **PRO TIP:** Automate security. Validate continuously. Stay compliant. Protect your systems.

– VERIQTA



YouTube
VERIQTA



GitHub
veriqta



LinkedIn
VERIQTA



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



ANSIBLE



PRODUCTION TROUBLESHOOTING

FIND. UNDERSTAND. FIX. PREVENT.

SYSTEMATIC TROUBLESHOOTING FOR RELIABLE AUTOMATION.

“Great automation is not about never failing.
It's about failing fast, finding the cause,
and making the system better.”

- VERIQTA ”

WHY TROUBLESHOOT?

- ✓ Reduce downtime and MTTR.
- ✓ Improve reliability and stability.
- ✓ Find root cause, not just symptoms.
- ✓ Prevent repeat incidents.
- ✓ Build trust in automation.
- ✓ Deliver consistent results.

1 VERBOSE LOGGING



- ✓ Increase verbosity to see detailed execution flow.
- ✓ Use `-v`, `-vv`, `-vvv` for more detail.
- ✓ Combine with `log_path` for persistent logs.

EXAMPLES

```
# Increase verbosity
ansible-playbook site.yml -v
ansible-playbook site.yml -vvv

# Log to file
ansible-playbook site.yml -vvv \
--log-path=/var/log/ansible/ansible.log
```

2 DEBUG MODULE



- ✓ Use debug module to inspect variables and data.
- ✓ Helpful for validating logic and conditions.
- ✓ Remove debug tasks in production.

EXAMPLE

```
- name: Debug variable
  debug:
    var: my_variable

- name: Debug complex data
  debug:
    var: hostvars[inventory_hostname]
  verbosity: 2
```

3 FAILED TASKS



- ✓ Read the error message carefully.
- ✓ Check the module, parameters, and target system.
- ✓ Re-run with `-vvv` for full details.
- ✓ Check previous task outputs.

COMMON CAUSES

- Invalid module parameters
- Missing files or paths
- Permission issues
- Service not running
- Network connectivity
- Incorrect inventory or variables

4 SSH FAILURES



- ✓ SSH is the transport engine.
- ✓ Failures stop all automation.
- ✓ Validate connectivity and authentication first.
- ✓ Check user, key, permissions and network.

TROUBLESHOOTING CHECKLIST

- Test: `ssh user@host`
- Verify SSH key and permissions
- Check `~/.ssh/config`
- Check remote firewall
- Validate inventory hostname/IP
- Ensure user has sudo if required

5 MODULE DEBUGGING



- ✓ Reproduce the task with minimal inputs.
- ✓ Check module documentation.
- ✓ Validate arguments and types.
- ✓ Use `-vvv` to see raw module request/response.

EXAMPLE: DEBUG MODULE INPUT/OUTPUT

```
TASK [Install nginx] *****
<host> ESTABLISH SSH CONNECTION FOR USER: ubuntu
<host> EXEC /bin/sh -c 'echo ~ubuntu && sleep 0'
<host> PUT /tmp/ansible-tmp-123/AnsiballZ_apt.py
<host> EXEC /bin/sh -c 'chmod u+x /tmp/ansible-.../
<host> EXEC /bin/sh -c 'usr/bin/python3 /tmp/ansible-...
<host> FAILED! => ["msg": "E: Unable to locate package"]
```

6 PERFORMANCE ISSUES



- ✓ Slow playbooks impact reliability.
- ✓ Identify bottlenecks and optimise.
- ✓ Use profiling and timing.
- ✓ Tune forks, fact gathering, and module usage.

COMMON BOTTLENECKS

- Too many forks
- Fact gathering on large inventories
- Slow modules (e.g., `shell`, `get_url`)
- Network latency
- Large files and transfers
- Repeated tasks and loops

7 ROOT CAUSE ANALYSIS



- ✓ Go beyond the error.
- ✓ Identify the real cause.
- ✓ Use logs, facts, and system data.
- ✓ Apply permanent fixes.

RCA APPROACH

1. Identify the problem
2. Collect facts and logs
3. Reproduce the issue
4. Analyse and isolate cause
5. Implement fix
6. Validate and monitor
7. Document and prevent

8 USEFUL CLI OPTIONS



- ✓ Use the right tools for the job.
- ✓ Combine options for deep inspection.
- ✓ Know your debugging toolbox.

OPTION	PURPOSE
<code>-v</code>	Increase verbosity
<code>-vv</code>	More detailed output
<code>-vvv</code>	Very verbose (debug level)
<code>--step</code>	Step through tasks
<code>--start-at-task</code>	Start from specific task
<code>--syntax-check</code>	Check playbook syntax
<code>--check</code>	Dry run (no changes)
<code>--diff</code>	Show changes (if supported)
<code>--limit host1,host2</code>	Run on specific hosts

9 LOGGING & REPORTS



- ✓ Centralize logs for easier analysis.
- ✓ Keep logs for auditing.
- ✓ Use callback plugins for reports.

EXAMPLE: `ansible.cfg`

```
[defaults]
log_path = /var/log/ansible/ansible.log
callback_whitelist = timer, profile_tasks
stdout_callback = yaml

# View logs
tail -f /var/log/ansible/ansible.log
```

10 TROUBLESHOOTING WORKFLOW



QUICK TIP

- Read errors fully.
- Use `-vvv`.
- Think logically.
- Test changes.
- Automate fixes.

★ PRO TIP: Great engineers don't avoid problems. They solve them systematically and make systems stronger.

- VERIQTA

YouTube
VERIQTAGitHub
veriqtaLinkedIn
VERIQTAX
@veriqtaInstagram
@veriqtaTelegram
@veriqta



ANSIBLE

VERIQTA



ENTERPRISE PLAYBOOK DESIGN

DESIGN ONCE. REUSE EVERYWHERE.
SCALABLE, MAINTAINABLE, COLLABORATIVE AUTOMATION.

“Great playbooks are not written once and forgotten. They are designed for reuse, scale, and long-term maintainability.”

— VERIQTA ”

WHY GOOD DESIGN MATTERS?

- ✓ Consistency across environments.
- ✓ Faster development and delivery.
- ✓ Easier maintenance and scaling.
- ✓ Reduced duplication and errors.
- ✓ Better collaboration and reviews.
- ✓ Enterprise-grade automation.

1 REUSABLE ARCHITECTURE



- ✓ Build once, reuse everywhere.
- ✓ Use roles, collections, and shared components.
- ✓ Keep logic generic, configurations flexible.
- ✓ Follow DRY (Don't Repeat Yourself) principles.

KEY PRINCIPLES

- Reuse roles and collections
- Centralize common tasks
- Parameterize everything
- Avoid hardcoding values
- Design for extensibility
- Document and version everything

2 MODULAR PLAYBOOKS



- ✓ Break large automation into small, focused components.
- ✓ Use roles, task files, and includes.
- ✓ Keep playbooks thin, logic in roles.
- ✓ One purpose per role.

MODULAR STRUCTURE

```
site.yml (orchestrator)
├── roles/
│   ├── common/
│   ├── web/
│   ├── db/
│   └── monitoring/
├── group_vars/
└── host_vars/
```

3 ENVIRONMENT SEPARATION



- ✓ Separate configurations by environment.
- ✓ Use inventory, vars, and secrets per environment.
- ✓ Avoid mixing environments.

ENVIRONMENT STRATEGY

Environment	Inventory	Vars	Secrets
dev	inventory/dev/	group_vars/dev/	vault_dev.yml
staging	inventory/staging/	group_vars/staging/	vault_staging.yml
prod	inventory/prod/	group_vars/prod/	vault_prod.yml

- Separate state files and backends
- Different credentials per environment
- Protect production with approvals

4 NAMING STANDARDS



- ✓ Consistent naming improves readability and scale.
- ✓ Follow clear, predictable conventions.
- ✓ Easier onboarding for new team members.

EXAMPLES

Playbooks: site.yml, deploy_app.yml
Roles: nginx_setup, db_backup
Task files: main.yml, install.yml
Variables: app_name, db_port
Templates: nginx.conf.j2
Handlers: restart_nginx

NAMING RULES

- Use lowercase with underscores
- Use descriptive and meaningful names
- Be consistent across the project
- Avoid abbreviations unless standard

5 REPOSITORY STRUCTURE



- ✓ Organized structure makes projects scalable.
- ✓ Logical layout for roles, inventories, docs, and pipelines.
- ✓ Easy navigation and maintenance.

RECOMMENDED STRUCTURE

```
ansible-enterprise/
├── ansible.cfg
├── inventories/
│   ├── dev/
│   ├── staging/
│   └── prod/
├── group_vars/
├── host_vars/
├── roles/
├── collections/
├── library/ (custom modules)
├── playbooks/
│   ├── site.yml
│   └── other_playbooks.yml
├── templates/
├── files/
├── vars/ (global vars)
├── docs/
├── tests/
├── .gitignore
├── README.md
└── requirements.yml
```

6 TEAM COLLABORATION



- ✓ Collaboration is key to enterprise success.
- ✓ Use Git workflows, code reviews, and CI/CD.
- ✓ Document and communicate changes.

COLLABORATION PRACTICES

- Use Git with feature branches
- Enforce code reviews
- Keep playbooks idempotent
- Write clear documentation
- Use CI/CD for testing and validation
- Communicate changes and ownership

GIT WORKFLOW EXAMPLE



USEFUL TOOLS

- Ansible Galaxy
- Molecule (Role testing)
- ansible-lint (Linting)
- yamllint (YAML Linting)
- CI/CD (GitHub Actions, GitLab CI, etc.)

7 ENTERPRISE PLAYBOOK DESIGN BEST PRACTICES



Design for idempotency



Keep playbooks small and focused



Use roles and collections



Parameterize everything



Secure secrets with Ansible Vault



Test before production



QUICK TIP
A well-designed playbook today saves hundreds of hours of maintenance tomorrow.

8 PLAYBOOK DESIGN WORKFLOW



1. Understand Requirements



2. Design Architecture



3. Build Modular Components



4. Integrate & Test



5. Validate in Environments



6. Deploy Safely



7. Monitor & Optimize

★ **PRO TIP:** Good design is the foundation of great automation. Keep it modular, secure, and consistent.

— VERIQTA



ANSIBLE

VERIQTA



PRODUCTION DEPLOYMENT STRATEGIES

DEPLOY SMART. VALIDATE CONTINUOUSLY. DELIVER SAFELY.
MINIMIZE RISK. MAXIMIZE RELIABILITY.

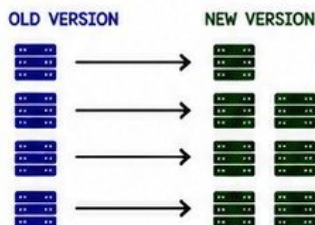
“A good deployment strategy is not just about getting new code to production, it is about controlling risk, validating success, and ensuring fast recovery. - VERIQTA”

WHY IT MATTERS?

- ✓ Reduce downtime and risk.
- ✓ Deliver changes continuously.
- ✓ Quickly detect and rollback issues.
- ✓ Improve reliability and trust.
- ✓ Align deployments with business.
- ✓ Ensure safe, repeatable releases.

1 ROLLING DEPLOYMENTS

- ✓ Update instances gradually.
- ✓ Old version and new version run at the same time.
- ✓ Zero downtime deployment.
- ✓ Automatic or manual batches.



HOW IT WORKS

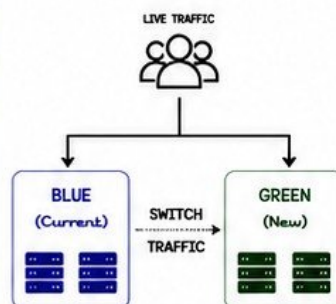
1. Select a batch of servers.
2. Drain connections.
3. Update to new version.
4. Health check.
5. Repeat for next batch.

BEST FORS

- Stateless applications
- Large environments
- When zero downtime is required

2 BLUE-GREEN DEPLOYMENT

- ✓ Two identical environments.
- ✓ Switch traffic from Blue to Green.
- ✓ Instant rollback by switching back.
- ✓ Requires more infrastructure.



HOW IT WORKS

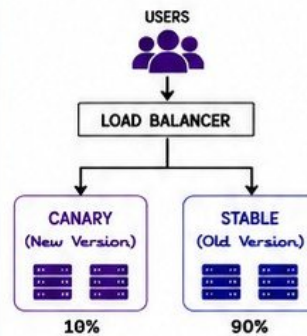
1. Deploy new version to Green.
2. Run full tests and validation.
3. Switch traffic to Green.
4. Monitor.
5. Keep Blue as fallback.

BEST FOR

- Critical applications
- Easy and fast rollback
- When speed and stability matter

3 CANARY DEPLOYMENT

- ✓ Release to a small subset first.
- ✓ Validate with real user traffic.
- ✓ Gradually increase traffic.
- ✓ Stop if issues detected.



HOW IT WORKS

1. Deploy new version to canary.
2. Send small % of traffic.
3. Monitor metrics and logs.
4. Increase traffic gradually.
5. Rollback if issues occur.

BEST FOR

- Microservices
- High-volume applications
- Risk-sensitive changes

4 VALIDATION

- ✓ Automated and manual tests.
- ✓ Health checks and smoke tests.
- ✓ Functional and integration tests.
- ✓ Performance and security tests.



HEALTH CHECKS



SMOKE TESTS



INTEGRATION TESTS



PERFORMANCE TESTS



SECURITY TESTS



USER VALIDATION

VALIDATION CHECKLIST

- Application starts successfully
- All critical endpoints respond
- Database and dependencies OK
- No error spikes in logs
- Performance within baseline
- Security scan passed
- User acceptance (if required)

BEST PRACTICES

- Automate validation
- Fail fast, rollback fast
- Monitor continuously

5 ROLLBACK STRATEGY

- ✓ Always have a rollback plan.
- ✓ Make rollbacks fast and automated.
- ✓ Keep previous versions accessible.
- ✓ Test rollback procedures regularly.

ROLLBACK OPTIONS

	Automatic Rollback	Triggered by failed health checks or monitoring alerts.
	Manual Rollback	Rollback when issues are confirmed by the team.
	Versioned Releases	Rollback by redeploying the previous stable version.
	Database Rollback	Use backups or migration rollbacks if schema changes fail.

ROLLBACK CHECKLIST



- Identify issue and impact
- Decide rollback approach
- Notify stakeholders
- Switch traffic (if needed)
- Rollback deployment
- Validate system health
- Communicate status
- Post-incident review

6 CHANGE MANAGEMENT

- ✓ Follow a structured change process.
- ✓ Ensure visibility, approval and traceability.
- ✓ Communicate before, during and after changes.

CHANGE MANAGEMENT PROCESS



CHANGE RECORD

- Change ID / Ticket
- Description
- Environment
- Owner
- Approval
- Start Time / End Time
- What Changed
- Rollback Plan
- Status
- Post-Change Notes

7 PRODUCTION DEPLOYMENT LIFECYCLE



QUICK TIP

- Start small, automate everything, monitor continuously, and always be ready to rollback.

★ PRO TIP: The best deployment strategy is the one that fits your application, team, and risk tolerance.

- VERIQTA



YouTube
VERIQTA



GitHub
veriqta



LinkedIn
VERIQTA



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



ANSIBLE

VERIQTA



ANSIBLE AUTOMATION PLATFORM

ENTERPRISE AUTOMATION. CENTRALIZED. SECURE. GOVERNED.
BUILT FOR SCALE AND RELIABILITY.

“ Ansible Automation Platform brings structure, governance, and scale to enterprise automation. It turns ad-hoc scripts into trusted, repeatable processes that deliver business value.

- VERIQTA ”

WHY IT MATTERS?

- ✓ Centralize automation at scale
- ✓ Enforce security and compliance
- ✓ Improve visibility and control
- ✓ Promote reuse and consistency
- ✓ Accelerate delivery safely
- ✓ Operate across hybrid environments

1 AUTOMATION CONTROLLER



- ✓ Centralized UI and API
- ✓ Manage inventories, credentials, projects, jobs, and schedules
- ✓ Role-based access
- ✓ Audit and activity logs
- ✓ Highly available (HA) and scalable

KEY COMPONENTS

- Web UI
- REST API
- Task Engine
- Database
- Redis
- Dispatcher

2 EXECUTION ENVIRONMENTS



- ✓ Container images with Ansible and dependencies
- ✓ Ensure consistency across environments
- ✓ Isolate Python dependencies
- ✓ Versioned and distributable
- ✓ Support for private registries

BEST PRACTICES

- Use minimal, secure images
- Scan images for vulnerabilities
- Version and tag images
- Test execution environments before production

3 JOB TEMPLATES



- ✓ Define automation jobs
- ✓ Reuse playbooks with parameters
- ✓ Limit choices and errors
- ✓ Survey prompts for user input
- ✓ Track job history and status

EXAMPLE FIELDS

- Name
- Project / Playbook
- Inventory
- Credentials
- Extra Variables
- Limit / Tags
- Options (forks, timeout, etc.)

4 SCHEDULES



- ✓ Run automation at specific times
- ✓ Cron-style scheduling
- ✓ Time zone support
- ✓ Enable recurring maintenance
- ✓ Use ad-hoc or template jobs

USE CASES

- Daily compliance checks
- Weekly patching
- Log rotation
- Certificate renewal
- Backups and cleanups

5 RBAC (ROLE-BASED ACCESS)



- ✓ Granular access control
- Organizations
- Teams
- Roles
- Users
- ✓ Object-level permissions
- ✓ Enforce least privilege

COMMON ROLES

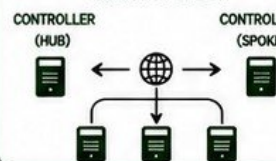
Admin	Full system access
System Auditor	Read-only, audit logs
Inventory Admin	Manage inventories
Job Admin	Manage and run jobs
Execution Admin	Manage execution envs
Credential Admin	Manage credentials
Project Admin	Manage projects

6 AUTOMATION MESH



- ✓ Extend automation across multiple Automation Controllers
- ✓ Share inventories, credentials, projects
- ✓ Distribute execution across sites and regions
- ✓ Improve resilience and scalability

MESH TOPOLOGY



7 ENTERPRISE GOVERNANCE



- ✓ Enforce policies and standards
- ✓ Audit all activities
- ✓ Track changes and approvals
- ✓ Ensure compliance and security
- ✓ Integrate with ITSM, SIEM, and Tower APIs

GOVERNANCE PILLARS

- Security & Compliance
- Audit & Reporting
- Standardization
- Change Management
- Data Protection

8 KEY INTEGRATIONS



- ✓ ITSM (ServiceNow, Jira)
- ✓ Cloud (AWS, Azure, GCP)
- ✓ Monitoring (Prometheus, Grafana, Datadog)
- ✓ SIEM (Splunk, ELK)
- ✓ Git (GitHub, GitLab)
- ✓ CMDB / Asset Mgmt

BENEFITS

- End-to-end automation
- Better visibility
- Faster incident response
- Governed change management
- Consistent operations

9 PLATFORM ARCHITECTURE OVERVIEW



USERS

AUTOMATION CONTROLLER

- Web UI
- REST API
- Task Engine
- Scheduler
- RBAC
- Audit

EXECUTION NODES

- Run jobs and tasks at scale

MANAGED NODES

- Linux / Windows / Network / Cloud

DATA STORES

- PostgreSQL (Database)
- Redis (Caching)
- File Storage



QUICK TIP

Start small, standardize your automation, govern access, and scale safely.

10 ANSIBLE AUTOMATION PLATFORM VALUE



SCALE

Automate across thousands of nodes reliably



SECURITY

Enforce RBAC, audit logs, and secure execution



RELIABILITY

Consistent execution with versioned environments



EFFICIENCY

Reuse, standardize, and eliminate manual work



COLLABORATION

Teams work together with governed automation



GOVERNANCE

Policies, approvals, and compliance built-in

★ PRO TIP: Governed automation is not just about tools; it's about standards, visibility, and trust.

- VERIQTA



YouTube
VERIQTA



GitHub
veriqta



LinkedIn
VERIQTA



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



ANSIBLE

VERIQTA



ENTERPRISE AUTOMATION PATTERNS

REPEATABLE PATTERNS. CONSISTENT OUTCOMES.
AUTOMATE AT SCALE WITH CONFIDENCE.

“Great automation is not just about writing playbooks;
it's about solving real business problems with
stable, secure, and scalable patterns.”

- VERIQTA ”

WHY PATTERNS MATTER?

- ✓ Solve common problems faster
- ✓ Ensure consistency and quality
- ✓ Reduce manual effort and errors
- ✓ Improve security and compliance
- ✓ Enable scalability and reuse
- ✓ Drive operational excellence

1 INFRASTRUCTURE PROVISIONING



Automate the end-to-end provisioning of infrastructure resources.

- ✓ Create networks, compute, storage, and security
- ✓ Use modules, roles, and collections
- ✓ Idempotent and repeatable
- ✓ Integrate with cloud APIs

BEST PRACTICES

- Use IaC principles
- Parameterize everything
- Use dynamic inventories
- Validate before provisioning
- Tag and document resources

USE CASES

- New application environments
- Disaster recovery setup
- Test/Dev environment creation
- Account/Subscription setup

2 CONFIGURATION MANAGEMENT



Maintain consistent configuration across servers, applications, and services.

- ✓ Enforce desired state
- ✓ Use roles and variables
- ✓ Manage packages, files, users, services
- ✓ Ensure baseline compliance

BEST PRACTICES

- Keep roles focused
- Use version control
- Use handlers and notify
- Test in non-prod first
- Document configuration baselines

USE CASES

- OS hardening
- Application setup
- Patch management
- Service configuration

3 COMPLIANCE AUTOMATION



Continuously assess and enforce policy, security, and compliance requirements.

- ✓ Run compliance checks
- ✓ Generate audit reports
- ✓ Enforce security baselines
- ✓ Evidence collection

BEST PRACTICES

- Map controls to frameworks
- Automate evidence collection
- Schedule regular scans
- Store reports securely
- Alert on non-compliance

USE CASES

- CIS benchmark enforcement
- PCI DSS compliance
- SOX automation
- ISO 27001 controls

4 DRIFT REMEDIATION



Detect configuration drift and automatically remediate to restore desired state.

- ✓ Detect drift using facts
- ✓ Compare with desired state
- ✓ Remediate automatically
- ✓ Log and alert changes

BEST PRACTICES

- Define desired state clearly
- Use check mode for validation
- Remediate in safe windows
- Maintain change history
- Exclude intentional exceptions

USE CASES

- Unauthorized changes
- Config file modifications
- Drift after manual changes
- Policy violations

5 SELF-HEALING SYSTEMS



Automatically detect issues and take action to restore services and maintain uptime.

- ✓ Monitor health and status
- ✓ Trigger remediation playbooks
- ✓ Restart services, replace nodes
- ✓ Escalate if needed

BEST PRACTICES

- Define clear health checks
- Use event-driven triggers
- Implement safe remediation
- Avoid flapping with cooldowns
- Escalate after retries

USE CASES

- Service failure recovery
- Node replacement
- Auto scaling groups
- Application self-recovery

6 MULTI-CLOUD AUTOMATION



Automate across multiple cloud providers with consistent standards and policies.

- ✓ Abstract cloud differences
- ✓ Use collections and modules
- ✓ Centralize credentials
- ✓ Enforce governance policies

BEST PRACTICES

- Use cloud-agnostic patterns
- Store credentials securely
- Tag and standardize resources
- Monitor cost and usage
- Enforce security baselines

USE CASES

- Hybrid cloud deployments
- Multi-cloud DR
- Cloud migration
- Consistent governance

ENTERPRISE AUTOMATION PATTERNS - CROSS-CUTTING PRINCIPLES



SECURITY FIRST

Encrypt secrets, use least privilege, and follow security best practices.



IDEMPOTENT AUTOMATION

Ensure playbooks can run multiple times with the same desired outcome.



REUSABLE & MODULAR

Build roles and collections for reuse across teams and projects.



OBSERVABILITY

Log everything, monitor continuously, and alert on issues.



COLLABORATION

Use version control, peer reviews, and clear documentation.



CONTINUOUS IMPROVEMENT

Refine playbooks, automate feedback, and iterate often.

★ PRO TIP: Start with proven patterns. Standardize. Automate. Monitor. Improve.

- VERIQTA

REAL PRODUCTION CASE STUDIES

REAL PROBLEMS. REAL AUTOMATION. REAL IMPACT.
AUTOMATION SOLVING BUSINESS-CRITICAL CHALLENGES.

WHY REAL CASE STUDIES?

- ✓ Learn from real-world scenarios
- ✓ Understand production challenges
- ✓ See automation in action
- ✓ Avoid common mistakes
- ✓ Apply patterns with confidence
- ✓ Deliver measurable value



ANSIBLE

“Automation is proven when it solves real problems at scale, consistently, and repeatably.

These are real challenges solved with Ansible. - VERIQTA”

1 FLEET PATCHING AUTOMATION



ENVIRONMENT

500+ Linux Servers
(Production)

TOOLS USED

Ansible Playbooks
AWX / Automation
Controller
Inventory Plugin

CHALLENGE

- Manual patching was slow, inconsistent, and caused downtime.

SOLUTION (ANSIBLE)

- Automated pre-patch backup and validation
- OS patching with reboot handling
- Post patch verification and service checks
- Automatic reporting and notifications

IMPACT

- Patching time reduced by 85%
- Zero downtime during patch window
- 100% compliance visibility
- Consistent patching across all servers

2 WEB SERVER DEPLOYMENT



ENVIRONMENT

Nginx Web Tier
(Production)

TOOLS USED

Ansible Playbooks
Jinja2 Templates
Git Repository

CHALLENGE

- Manual deployments were error-prone and inconsistent across environments.

SOLUTION (ANSIBLE)

- Code-based web server provisioning
- Auto configuration with templates
- SSL, firewall, and security hardening
- Blue-Green deployment for zero downtime

IMPACT

- Deployment time reduced by 90%
- Zero downtime deployments
- Standardized and repeatable builds
- Improved security posture

3 KUBERNETES CLUSTER AUTOMATION



ENVIRONMENT

Multi-Node K8s Cluster
(Production)

TOOLS USED

Ansible Playbooks
kubeadm
Jinja2 Templates

CHALLENGE

- Cluster setup and scaling were manual and time consuming.

SOLUTION (ANSIBLE)

- Automated cluster provisioning with kubeadm
- Worker node join automation
- CNI, DNS, Ingress, and monitoring setup
- Node lifecycle management

IMPACT

- Cluster setup reduced from hours to minutes
- Consistent, repeatable cluster builds
- Easy scaling and lifecycle management
- Reduced human errors

4 DISASTER RECOVERY AUTOMATION



ENVIRONMENT

Critical Applications
(Production)

TOOLS USED

Ansible Playbooks
AWX / Controller
Cloud APIs

CHALLENGE

- DR procedures were manual and recovery time was high.

SOLUTION (ANSIBLE)

- Automated DR environment provisioning
- Database restore and validation
- Application failover automation
- Full DR runbook automation

IMPACT

- Recovery time reduced by 70%
- Consistent DR across environments
- Regular DR testing made easy
- Higher business continuity

5 CONFIGURATION DRIFT REMEDIATION



ENVIRONMENT

Hybrid Infrastructure
(Cloud + On-Prem)

TOOLS USED

Ansible Playbooks
AWX / Controller
Compliance Modules

CHALLENGE

- Configuration drift caused compliance issues and outages.

SOLUTION (ANSIBLE)

- Scheduled compliance scans
- Detect drift using Ansible facts
- Auto-remediate to desired state
- Reporting and alerting

IMPACT

- Drift issues reduced by 95%
- Continuous compliance maintained
- Fewer incidents and outages
- Audit-ready infrastructure

6 SECURITY REMEDIATION AUTOMATION



ENVIRONMENT

Enterprise Servers
(Production)

TOOLS USED

Ansible Playbooks
Security Collections
AWX / Controller

CHALLENGE

- Security vulnerabilities were not remediated quickly.

SOLUTION (ANSIBLE)

- Automated vulnerability scans
- Patch and configuration remediation
- CIS hardening enforcement
- Security compliance reporting

IMPACT

- Vulnerability exposure reduced by 90%
- Faster remediation and response
- Stronger security posture
- Better audit and compliance scores

KEY LESSONS

- Automation turns manual tasks into reliable systems.
- Consistency and repeatability reduce risks.
- Visibility and reporting drive continuous improvement.
- Test automation regularly for trust and reliability.

BUSINESS IMPACT SUMMARY



80%+
Reduction in
Manual Effort



95%+
Improvement in
Compliance



70%+
Reduction in
Recovery Time



90%+
Increase in
Operational
Efficiency

PLAYBOOK FOR SUCCESS

- ✓ Automate small, then scale.
- ✓ Use version control for everything.
- ✓ Test in non-production first.
- ✓ Monitor and improve continuously.
- ✓ Document and share knowledge.

★ PRO TIP: Real automation is measured by the problems it solves and the value it delivers.

- VERIQTA



YouTube
VERIQTA



GitHub
veriqta



LinkedIn
VERIQTA



X
@veriqta



Instagram
@veriqta



Telegram
@veriqta



ANSIBLE



SENIOR ANSIBLE INTERVIEW TOPICS

DEEP KNOWLEDGE. REAL EXPERIENCE. ENTERPRISE IMPACT.
BE THE ENGINEER WHO DESIGNS, AUTOMATES, AND LEADS.

“Senior engineers don't just write playbooks; they design scalable automation systems that are secure, reliable, performant, and easy to maintain.

– VERIQTA ”

WHAT INTERVIEWERS LOOK FOR

- ✓ Deep technical knowledge
- ✓ Real-world problem solving
- ✓ Scalable automation design
- ✓ Security and governance mindset
- ✓ Performance and reliability focus
- ✓ Clear communication

1 IDEMPOTENCY



Ensure playbooks can be run multiple times without changing the system unnecessarily.

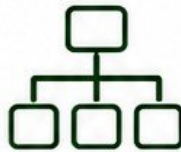
KEY POINTS

- Idempotent modules and states
- Avoid 'command' and 'shell' when possible
- Use creates/removes, unless, and when
- Check mode and diff mode
- Design tasks that report accurate changes

EXAMPLE QUESTION

How do you ensure idempotency in a playbook that installs a package and updates a file?

2 PLAYBOOK ARCHITECTURE



Design maintainable, modular, reusable, and scalable playbooks for complex environments.

KEY POINTS

- Role-based design and best practices
- Separation of concerns (roles, vars, tasks)
- Collections and reusable content
- Dynamic inventories and groups
- Include vs Import, tags, and blocks

EXAMPLE QUESTION

How would you structure playbooks for a multi-tier application across environments?

3 VARIABLE PRECEDENCE



Understand how Ansible resolves variables and how to avoid unexpected overrides.

KEY POINTS

- Know the full variable precedence order
- Extra vars vs role defaults vs group vars
- Host vars, facts, and set_facts
- Variable scope and lifetime
- Best practices for variable management

EXAMPLE QUESTION

Why is my variable not taking effect? Explain Ansible variable precedence.

4 PERFORMANCE OPTIMIZATION



Write faster playbooks that scale across thousands of hosts reliably.

KEY POINTS

- Forks, serial, and strategy tuning
- Async tasks and poll
- Fact caching and selective gathering
- Reduce unnecessary SSH connections
- Optimize loops, lookups, and templates

EXAMPLE QUESTION

How would you optimize a playbook that takes too long to run across 5,000 servers?

5 TROUBLESHOOTING SCENARIOS



Diagnose and fix real production issues in Ansible-driven environments.

KEY POINTS

- Verbose modes (-v, -vvv) and debug
- Callback plugins and logging
- Check mode and diff for analysis
- Common failure patterns and fixes
- Use of rescue blocks and error handling

EXAMPLE QUESTION

A playbook works on dev but fails on prod. How do you troubleshoot and resolve it?

6 ENTERPRISE DESIGN QUESTIONS



Design enterprise-grade automation solutions that are secure, compliant, and future-proof.

KEY POINTS

- Multi-team automation at scale
- RBAC, credentials, and security
- Governance, compliance, and audit
- CI/CD integration for automation
- High availability and disaster recovery

EXAMPLE QUESTION

Design an enterprise automation platform using Ansible Automation Platform.

BONUS: OTHER IMPORTANT INTERVIEW AREAS



SECURITY

- Vault and secret management
- Least privilege access
- SSH hardening



CLOUD AUTOMATION

- AWS, Azure, GCP modules
- Dynamic inventories
- Cloud native patterns



MODULES & PLUGINS

- Custom modules
- Lookup & filter plugins
- Callback plugins



MONITORING & LOGGING

- Centralized logging
- Alerting and notifications
- Observability patterns



CI/CD INTEGRATION

- GitOps with Ansible
- Pipeline automation
- Quality gates & testing

★ PRO TIP: Practice real-world scenarios. Build. Break. Fix. Explain. That's how senior engineers think.

– VERIQTA



ANSIBLE

VERIQTA



PRODUCTION READINESS CHECKLIST

VALIDATE. SECURE. TEST. MONITOR. GOVERN.
AUTOMATE WITH CONFIDENCE.

“ Production readiness is not a step.
It is a standard of excellence. - VERIQTA ”

GOAL

Ensure your Ansible automation is reliable, secure, tested, observable, and ready for production at enterprise scale.

1 PLAYBOOK VALIDATION



- ☐ Syntax check (ansible-lint)
- ☐ Idempotency verified
- ☐ No hardcoded values
- ☐ Proper error handling
- ☐ Tasks are modular & reusable
- ☐ Handlers used correctly
- ☐ Check mode tested (--check)
- ☐ Tags and blocks validated

2 INVENTORY VERIFICATION



- ☐ All hosts reachable
- ☐ Correct groups & variables
- ☐ No duplicate hosts
- ☐ Dynamic inventory tested
- ☐ Host variables validated
- ☐ Group hierarchy verified
- ☐ Connection settings tested
- ☐ Limit patterns verified

3 SECRETS PROTECTION



- ☐ No plaintext passwords
- ☐ Vault/encryption enabled
- ☐ Secrets in external vault
- ☐ Least privilege access
- ☐ Credentials rotated
- ☐ Env variables secured
- ☐ API tokens protected
- ☐ Audit access to secrets

4 TESTING STRATEGY



- ☐ Unit tests (molecule/pytest)
- ☐ Integration tests done
- ☐ Check mode validation
- ☐ Staging environment tests
- ☐ Negative scenario tests
- ☐ Performance tests (scale)
- ☐ Security/compliance tests
- ☐ User acceptance tests

5 ROLLBACK PLAN



- ☐ Version control (Git)
- ☐ Previous version tagged
- ☐ Rollback playbook available
- ☐ Backup before changes
- ☐ Database rollback plan
- ☐ Service restart strategy
- ☐ Quick rollback documented
- ☐ Rollback tested successfully

6 MONITORING INTEGRATION



- ☐ Logs centralized
- ☐ Ansible job logging enabled
- ☐ Alerts configured
- ☐ Metrics collected
- ☐ Integrate with APM/Monitoring
- ☐ Health checks implemented
- ☐ Notifications configured
- ☐ Observability dashboard ready

7 DOCUMENTATION



- ☐ Playbook purpose documented
- ☐ Usage instructions provided
- ☐ Variables documented
- ☐ Dependencies listed
- ☐ Runbook created
- ☐ Change log maintained
- ☐ Examples & scenarios
- ☐ Documentation up to date

8 AUTOMATION GOVERNANCE



- ☐ RBAC enforced
- ☐ Code review process
- ☐ Approval workflow
- ☐ Change management followed
- ☐ Compliance policies enforced
- ☐ Audit logging enabled
- ☐ Separation of duties
- ☐ Governance framework aligned

9 FINAL PRODUCTION CHECKLIST



- ✓ All playbooks validated and tested
- ✓ Inventory verified and up to date
- ✓ Secrets secured and access controlled
- ✓ Backups completed
- ✓ Rollback plan tested and documented
- ✓ Monitoring and alerts active
- ✓ Documentation complete and reviewed
- ✓ Security and compliance verified
- ✓ Stakeholder sign-off obtained
- ✓ Ready for production deployment

10 SENIOR ENGINEER PRINCIPLES



- ✓ **AUTOMATE REPEATABILITY** - Build systems that run the same way every time.
- ✓ **DESIGN FOR FAILURE** - Expect failures and handle them gracefully.
- ✓ **MAKE IT OBSERVABLE** - If you cannot see it, you cannot fix it.
- ✓ **SECURE BY DEFAULT** - Security is non-negotiable.
- ✓ **DOCUMENT EVERYTHING** - Future you (or your team) will thank you.
- ✓ **CONTINUOUS IMPROVEMENT** - Measure, learn, and make automation better.
- ✓ **THINK SCALE** - Design today for the scale of tomorrow.
- ✓ **OWNERSHIP & ACCOUNTABILITY** - Take responsibility for outcomes, not just tasks.



PRODUCTION READINESS = RELIABILITY + SECURITY + OBSERVABILITY + GOVERNANCE

When every box is checked, you are not just running automation.

You are running production-grade engineering.

★ **PRO TIP:** Test in staging like production. Automate with discipline. Monitor everything. Continuously improve.

- VERIQTA